



GSFC
UNIVERSITY
EDUCATION RE-ENVISIONED

Name of student	Prajapati Dev
Enrollment Number	25MSC04007
Course Name	Statistics II
Course Code	MSDS216
School	School of Science
Program	M.Sc. Data Science
Semester	II
Submission Date	



CERTIFICATE



This is to certify that the work entered in this laboratory journal is the work of Mr. Dev Prajapati of 2025 - 26 Year MSc, Discipline, Data Science Enrollment No. 25msc04007 and has satisfactorily completed the required number of experiments for the 2nd Semester, Academic Year 2025 – 26 in the Statistics-II Laboratory as laid down by the University.

Date:

Course In-charge



SR.NO	INDEX	PAGE NO.	DATE	SIGN
1	Practical List – 1			
	1.1 Formulate null and alternative hypothesis for the analysis of the real-world problem		07-01-26	
	1.2 Use R functions to calculate the probability of Type – I error and Type – II error and to Calculate the power of the test		07-01-26	
	1.3 Generate sampling distributions using R.		21-01-26	
	1.4 Calculate standard error for sample means and proportions		21-01-26	
	1.5 Visualize sampling distributions and standard error in R.		28-01-26	
2	Practical List - 2			
	2.1 Perform two-tailed and one-tailed tests using R functions.		04-02-26	
	2.2 Conduct Large Sample Tests for a Population mean & Population proportions in R.		04-02-26	
	2.3 Use Small Sample Tests for a population mean in R.		04-02-26	
	2.4 Comparison of mean for two dependent samples (Paired t - test) in R.		11-02-26	
	2.5 To calculate and Interpret p-values and make decisions		11-02-26	
3	Practical List – 3			
	3.1 Conduct chi-square test for population variance in R.		18-02-26	
	3.2 Test for ratio of two population variance using R		18-02-26	



	functions.			
	3.3 Use R functions to perform chi-square tests for independence and goodness of fitness.		11-03-26	
	3.4 Interpret test results and assess the goodness of fit in R.		11-03-26	
4	Practical List - 4			
	4.1 Visualize bivariate data using plots and graphs in R		18-03-26	
	4.2 To calculate coefficient of correlation using various methods and to interpret its value in R programming.		18-03-26	
	4.3 Practical exercises to simple linear regression in R		18-03-26	
	4.4 To test of significance of parameters fitted simple linear regression model in R.		25-03-26	
	4.5 Using R functions fit various non-linear regression models through transformation of variables.		25-03-26	
	4.6 Analyzing and presenting results from real-world datasets in R.		25-03-26	
5	Practical List - 5			
	5.1 Visualize multivariate data using plots and graphs in R.		01-04-26	
	5.2 Practical exercises to simple linear regression applying the matrix approach.		01-04-26	
	5.3 To test of significance of parameters fitted simple linear regression model		08-04-26	
	5.4 Practical applications of the matrix approach in multiple regression		15-04-26	
	5.5 Analyzing and presenting results from real-world datasets in R.		22-04-26	

PRACTICAL LIST - 1

1.1 Formulate null and alternative hypotheses for the analysis of real-world datasets.

1.2 Use R functions to calculate the probability of Type – I error and Type – II error and to Calculate the power of the test

1.3 Generate sampling distributions using R.

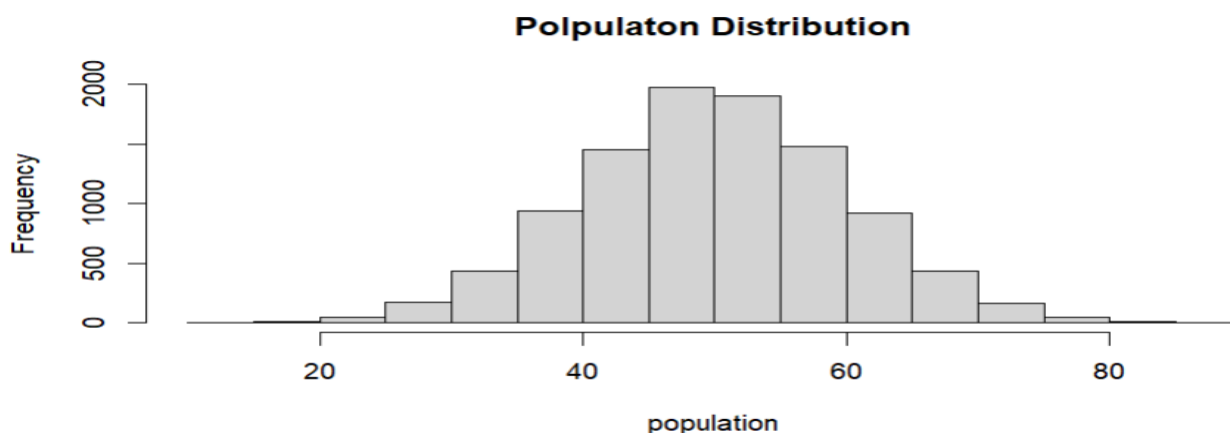
1.4 Calculate standard error for sample means and proportions

1.5 Visualize sampling distributions and standard error in R.

Code :

```
# Visualising Sampling Distribution and Standard Error in R
# Step - 1: Generate Population
set.seed(123)
population = rnorm(10000, mean = 50, sd = 10)
hist(population, main = "Polpulation Distribution", col = "lightgray")
```

Output - 1 :



```
# Step - 2: Generate Sampling Distribution
```

```
n = 30
num_samples = 1000
sample_means = replicate(num_samples, mean(sample(population, n)))
```

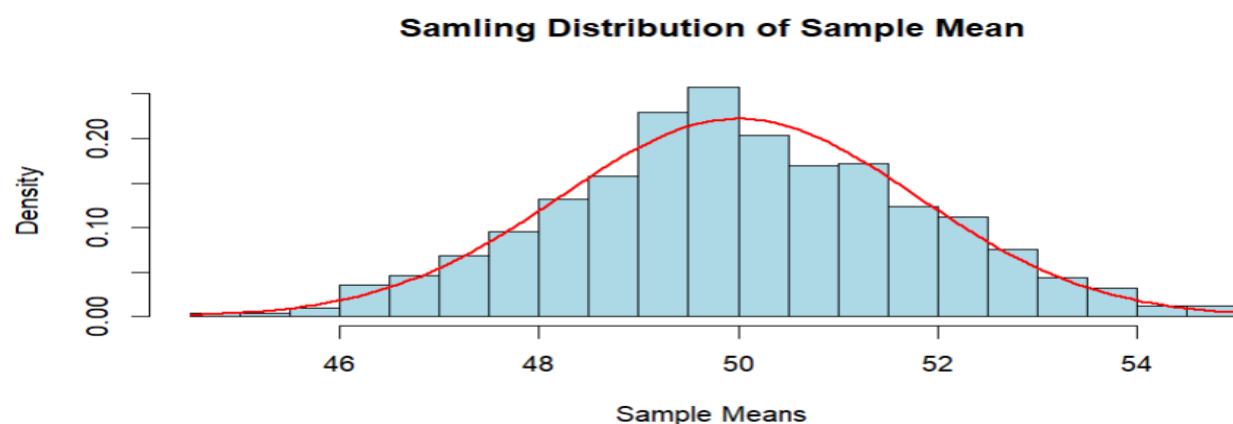
Step - 3: Visualize Sampling Distribution

```
hist(sample_means, breaks = 30, probability = TRUE, main = "Samling Distribution of Sample Mean", col = "lightblue", xlab = "Sample Means")
```

Step - 4: Add Normal Curve

```
curve(dnorm(x,mean = mean(sample_means),sd = sd(sample_means)),add = TRUE,col = "red",lwd = 2)
```

Output - 2 :



Step - 5: Calculate and Display Standard Error

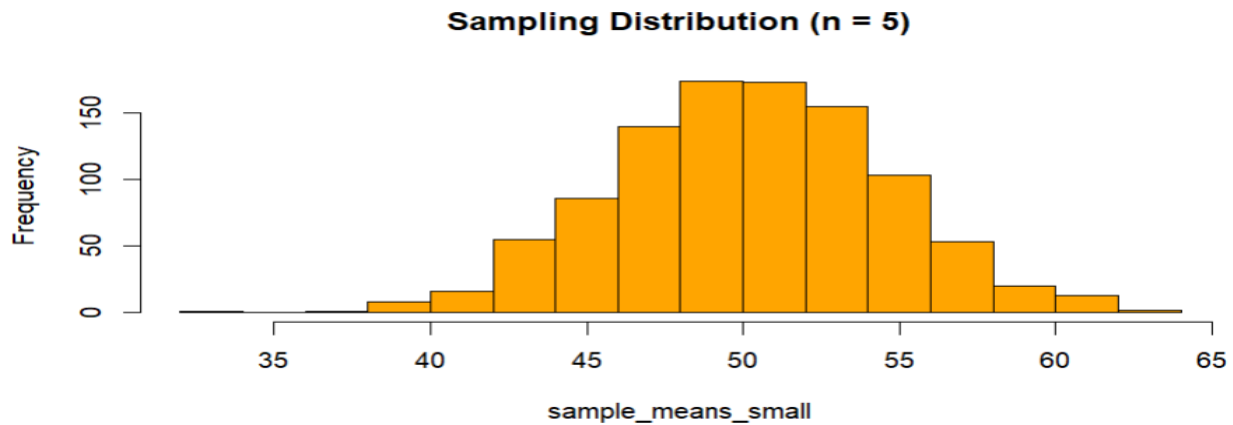
```
SE_theoretical = 10 / sqrt(n)
SE_simulated = sd(sample_means)
SE_theoretical
[1] 1.825742
SE_simulated
[1] 1.794127
```

Effect of Sample Size on SE

Small Sample

```
sample_means_small = replicate(1000,mean(sample(population,5)))
hist(sample_means_small,
main = "Sampling Distribution (n = 5)",col = "orange")
```

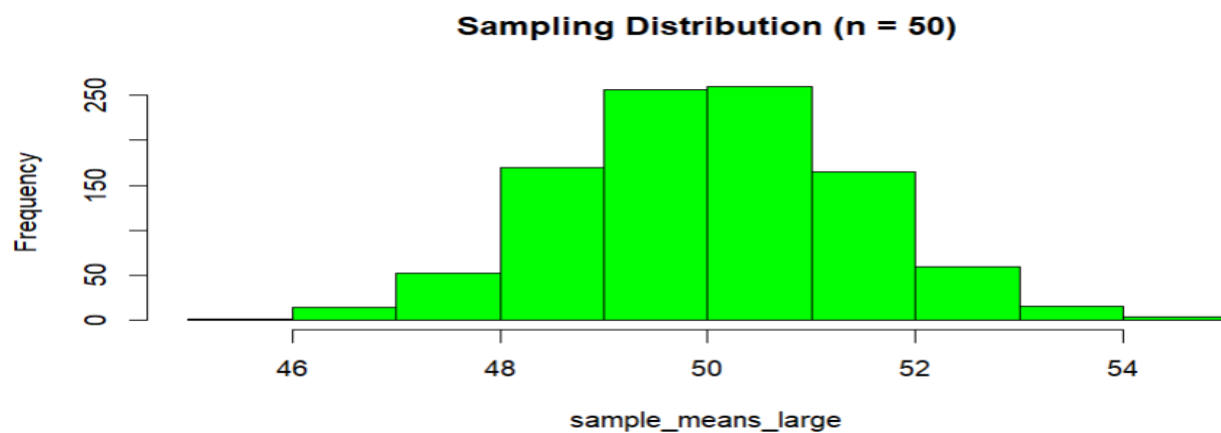
Output - 3 :



Large Sample

```
sample_means_large = replicate(1000,mean(sample(population,50)))
hist(sample_means_large,main = "Sampling Distribution (n = 50)",col = "green")
```

Output - 4 :

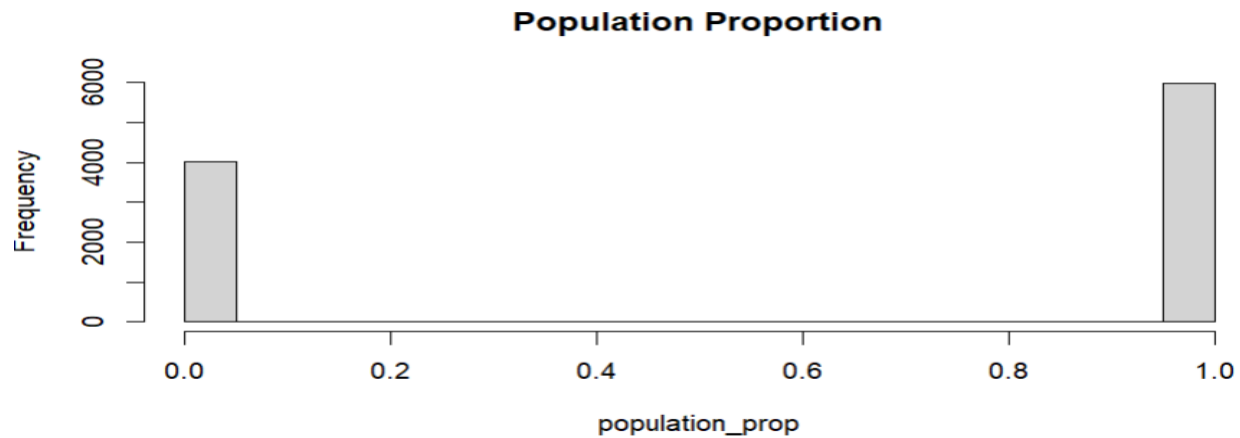


```
sd(sample_means_small)
[1] 4.337129
sd(sample_means_large)
[1] 1.423629
```

```
# Sampling Distribution of Proportion
# Create Binary Population
```

```
population_prop = rbinom(10000, size = 1, prob = 0.6)
hist(population_prop,main = "Population Proportion")
```

Output - 5 :

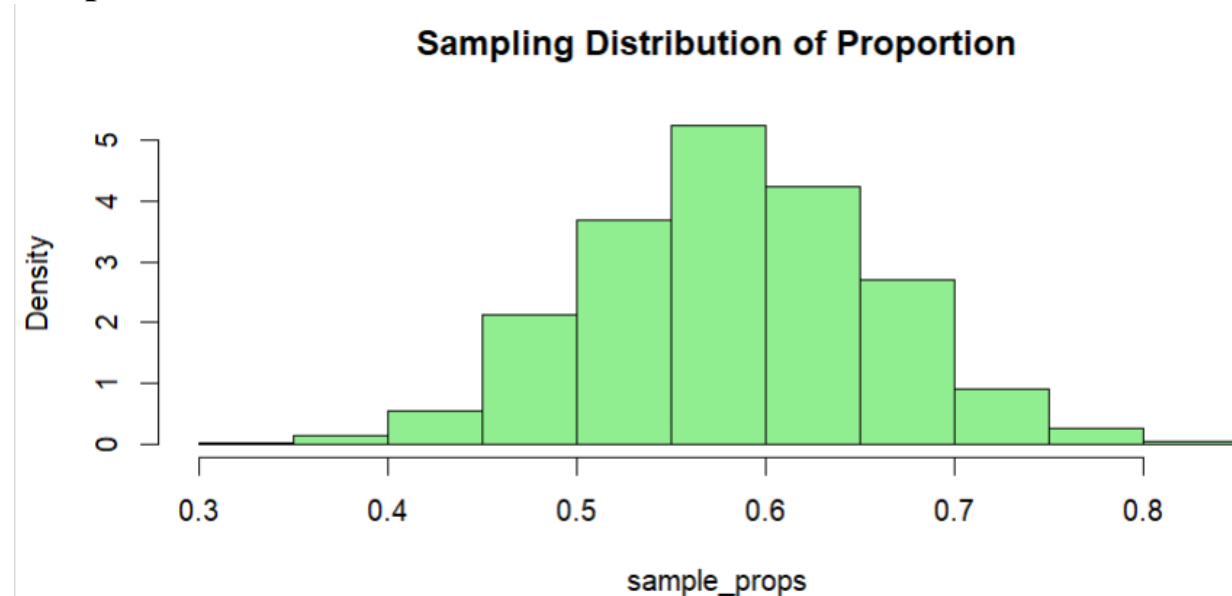


Generate Sample Proportions

n = 40

```
sample_props = replicate(1000, mean(sample(population_prop,n)))
hist(sample_props,probability = TRUE,main = "Sampling Distribution of Proportion",col =
"lightgreen")
```

Output - 6 :



Calculate SE of Proportion



```
p = 0.6
SE_prop_theoretical = sqrt(p*(1-p)/n)
SE_prop_simulated = sd(sample_props)
```

```
SE_prop_theoretical
[1] 0.07745967
SE_prop_simulated
[1] 0.07738397
```

```
# Two-tailed and One-Tailed Tests in R
```

```
# Scenario - 1
marks = c(65, 72, 68, 75, 70, 69, 80, 74, 66, 71)
```

```
# Step - 1: Two-Tailed Test
```

```
t.test(marks, mu = 70, alternative = "two.sided")
```

One Sample t-test

```
data: marks
t = 0.70321, df = 9, p-value = 0.4997
alternative hypothesis: true mean is not equal to 70
95 percent confidence interval:
 67.7831 74.2169
sample estimates:
mean of x
      71
```

```
# Step - 2: Right-Tailed Test
```

```
t.test(marks, mu = 70, alternative = "greater")
```

One Sample t-test

```
data: marks
t = 0.70321, df = 9, p-value = 0.2499
alternative hypothesis: true mean is greater than 70
95 percent confidence interval:
 68.39322      Inf
sample estimates:
mean of x
```



71

Step - 3: Left-Tailed Test

```
t.test(marks, mu = 70, alternative = "less")
```

One Sample t-test

```
data: marks
t = 0.70321, df = 9, p-value = 0.7501
alternative hypothesis: true mean is less than 70
95 percent confidence interval:
 -Inf 73.60678
sample estimates:
mean of x
      71
```

Scenario - 2

```
classA = c(65, 70, 75, 68, 72)
classB = c(60, 62, 70, 65, 66)
```

Two-Tailed Test

```
t.test(classA,classB,alternative = "two.sided")
```

Welch Two Sample t-test

```
data: classA and classB
t = 2.2307, df = 7.9992, p-value = 0.05624
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.1823468 10.9823468
sample estimates:
mean of x mean of y
      70.0      64.6
```

Right-Tailed Test

```
t.test(classA,classB,alternative = "greater")
```



Welch Two Sample t-test

data: classA and classB
t = 2.2307, df = 7.9992, p-value = 0.02812
alternative hypothesis: true difference in means is greater than 0
95 percent confidence interval:
0.8984496 Inf
sample estimates:
mean of x mean of y
70.0 64.6

Left-Tailed Test

```
t.test(classA,classB,alternative = "less")
```

Welch Two Sample t-test

data: classA and classB
t = 2.2307, df = 7.9992, p-value = 0.9719
alternative hypothesis: true difference in means is less than 0
95 percent confidence interval:
-Inf 9.90155
sample estimates:
mean of x mean of y
70.0 64.6

Scenario - 3

#Out of 200 students, 120 passed.

#Test if pass rate differs from 50%.

Two-Tailed Test

```
prop.test(120, 200, p = 0.5, alternative = "two.sided")
```

1-sample proportions test with continuity correction

data: 120 out of 200, null probability 0.5
X-squared = 7.605, df = 1, p-value = 0.005821
alternative hypothesis: true p is not equal to 0.5
95 percent confidence interval:
0.5283160 0.6677775
sample estimates:
p



0.6

Right-Tailed Test

```
prop.test(120, 200, p = 0.5, alternative = "greater")
```

1-sample proportions test with continuity correction

data: 120 out of 200, null probability 0.5

X-squared = 7.605, df = 1, p-value = 0.00291

alternative hypothesis: true p is greater than 0.5

95 percent confidence interval:

0.5395277 1.0000000

sample estimates:

p

0.6

Left-Tailed Test

```
prop.test(120, 200, p = 0.5, alternative = "less")
```

1-sample proportions test with continuity correction

data: 120 out of 200, null probability 0.5

X-squared = 7.605, df = 1, p-value = 0.9971

alternative hypothesis: true p is less than 0.5

95 percent confidence interval:

0.0000000 0.6576866

sample estimates:

p

0.6

Large Sample Test for Population Mean

Step - 1: Define Values in R

xbar = 520

mu0 = 500

sigma = 50

n = 100

alpha = 0.05

Step - 2: Calculate Standard Error

SE = sigma / sqrt(n)

SE



```
[1] 5
```

```
# Step - 3: Calculate Z-Statistic
```

```
z = (xbar - mu0) / SE
```

```
z
```

```
[1] 4
```

```
# Step - 4: Calculate p-value (Two-Tailed)
```

```
p_value = 2*(1-pnorm(abs(z)))
```

```
p_value
```

```
[1] 6.334248e-05
```

```
# step - 5: Decision
```

```
if(p_value < alpha){  
  print("Reject the Null Hypothesis")}
```

```
else{  
  print("Accept the Null Hypothesis")}
```

```
[1] "Reject the Null Hypothesis"
```

```
# Large Sample Test for Population Proportion
```

```
# Step - 1: Define Values
```

```
n = 400
```

```
x = 240
```

```
p0 = 0.5
```

```
 $\alpha$  = 0.05
```

```
# Step - 2: Calculate Sample Proportion
```

```
p_hat = x / n
```

```
p_hat
```

```
[1] 0.6
```

```
# Step - 3: Calculate SE
```

```
SE = sqrt(p0*(1 - p0) / n)
```

```
SE
```

```
[1] 0.025
```

```
# Step - 4: Calculate Z-Statistic
```

```
z = (p_hat - p0) / SE
```



```
z  
[1] 4
```

```
# Step - 5: Calculate p-value  
p_value = 2*(1 - pnorm(abs(z)))  
p_value  
[1] 6.334248e-05
```

```
# Step - 6: Decision
```

```
if(p_value < alpha){  
  print("Reject the Null Hypothesis")  
else{  
  print("Accept the Null Hypothesis")  
}  
[1] "Reject the Null Hypothesis"
```

```
# For Mean (Large Sample)  
t.test(data,mu = 50)
```

One Sample t-test

```
data: data  
t = -2.3238, df = 3, p-value = 0.1027  
alternative hypothesis: true mean is not equal to 50  
95 percent confidence interval:  
 14.4574 55.5426  
sample estimates:  
mean of x  
 35
```

```
# For Proportion  
prop.test(240, 400, p = 0.5, alternative = "two.sided")
```

1-sample proportions test with continuity correction

```
data: 240 out of 400, null probability 0.5  
X-squared = 15.602, df = 1, p-value = 7.815e-05  
alternative hypothesis: true p is not equal to 0.5  
95 percent confidence interval:  
 0.5499965 0.6480518  
sample estimates:  
 p  
0.6
```

PRACTICAL LIST – 2

- 2.1 Perform two-tailed and one-tailed tests using R functions.**
- 2.2 Conduct Large Sample Tests for a Population mean & Population proportions in R.**
- 2.3 Use Small Sample Tests for a population mean in R.**
- 2.4 Comparison of mean for two dependent samples (Paired t - test) in R.**
- 2.5 To calculate and Interpret p-values and make decisions**

Practical: One-Tailed and Two-Tailed Tests using R

Step 1: Create Large Data-set in R

Generate real-life like salary data

set.seed(123)

salary = rnorm(200, mean = 52000, sd = 8000)

salary

```
[1] 47516.19 50158.58 64469.67 52564.07 53034.30 65720.52 55687.33 41879.51 46505.18
[10] 48434.70 61792.65 54878.51 55206.17 52885.46 47553.27 66295.31 55982.80 36267.06
[19] 57610.85 48217.67 43457.41 50256.20 43791.96 46168.87 46999.69 38506.45 58702.30
[28] 53226.98 42894.90 62030.52 55411.71 49639.43 59161.01 59025.07 58572.65 57509.12
[37] 56431.34 51504.71 49552.30 48956.23 46442.34 50336.66 41876.83 69351.65 61663.70
[46] 43015.13 48776.92 48266.76 58239.72 51333.05 54026.55 51771.63 51657.04 62948.82
[55] 50193.83 64131.76 39609.98 56676.91 52990.83 53727.53 55037.12 47981.41 49334.34
[64] 43851.40 43425.67 54428.23 55585.68 52424.03 59378.14 68400.68 48071.75 33526.65
[73] 60045.91 46326.39 46495.93 60204.57 49721.82 42234.26 53450.43 50888.87 52046.11
[82] 55082.24 49034.72 57155.01 50236.11 54654.26 60774.71 55481.45 49392.55 61190.46
[91] 59948.03 56387.18 53909.85 46976.75 62885.22 47197.92 69498.66 64260.89 50114.40
[100] 43788.63 46316.75 54055.07 50026.46 49219.66 44387.05 51639.78 45720.76 38656.46
[109] 48958.19 59351.97 47397.22 56863.71 39056.94 51555.50 56155.26 54409.23 52845.41
[118] 46874.35 45202.37 43806.97 52941.17 44420.20 48075.54 49951.26 66750.90 46784.40
[127] 53883.09 52623.69 44305.15 51429.54 63556.41 55612.03 52329.86 48620.03 35574.02
[136] 61050.70 40314.88 57919.58 67272.83 40448.85 57614.27 49902.42 39422.85 39882.66
[145] 39187.71 47752.75 40305.96 57503.33 68800.87 41703.76 58301.91 58152.34 54657.62
[154] 43932.99 51044.38 49756.84 56503.92 49020.49 59815.79 49003.35 60421.69 43606.58
[163] 41918.76 77928.32 48665.14 54385.82 57092.56 48129.75 56134.90 54951.72 50276.96
[172] 52522.34 51727.46 69027.62 46069.31 43232.03 52302.31 54483.85 55492.19 48333.08
[181] 43493.39 62105.48 49202.80 45075.90 50109.76 50422.59 60879.36 52677.90 58032.43
[190] 48005.66 53715.56 49402.51 52756.67 44837.09 41513.59 67977.71 56805.67 41989.83
```



```
[199] 47110.67 42516.16
```

```
# Step 2: Explore the Data-set
```

```
summary(salary)
```

```
  Min. 1st Qu.  Median    Mean 3rd Qu.   Max.
 33527  46994  51530  51931  56547  77928
```

```
mean(salary)
```

```
[1] 51931.44
```

```
sd(salary)
```

```
[1] 7545.279
```

```
length(salary)
```

```
[1] 200
```

```
# Step 3: Two-Tailed Test
```

```
# Problem
```

```
#Test whether the average salary differs from ₹50,000
```

```
#Hypothesis
```

```
#H0 :  $\mu = 50000$ 
```

```
#H1 :  $\mu \neq 50000$ 
```

```
t.test(salary, mu = 50000, alternative = "two.sided")
```

One Sample t-test

```
data: salary
```

```
t = 3.6201, df = 199, p-value = 0.0003736
```

```
alternative hypothesis: true mean is not equal to 50000
```

```
95 percent confidence interval:
```

```
 50879.33 52983.54
```

```
sample estimates:
```

```
mean of x
```

```
 51931.44
```

```
#Interpretation
```

```
#Since p-value < 0.05 We reject the null hypothesis.
```

```
#Conclusion: The average salary is significantly different from ₹50,000.
```

```
# Step 4: One-tailed Test(Greater)
```

```
#Problem
```

```
#Test whether average salary is greater than ₹50,000
```

```
#Hypothesis
```

```
#H0 :  $\mu = 50000$ 
```

```
#H1 :  $\mu > 50000$ 
```

```
t.test(salary, mu = 50000, alternative = "greater")
```

One Sample t-test

```
data: salary
t = 3.6201, df = 199, p-value = 0.0001868
alternative hypothesis: true mean is greater than 50000
95 percent confidence interval:
 51049.75      Inf
sample estimates:
mean of x
51931.44
```

```
#Interpretation
#Since p-value > 0.05 We reject Ho
#Conclusion:The average salary is greater than ₹50,000.
```

```
#Step 5: One-Tailed Test (Less)
#Hypothesis
#Ho :  $\mu = 50000$ 
#H1 :  $\mu < 50000$ 
t.test(salary, mu = 50000, alternative = "less")
```

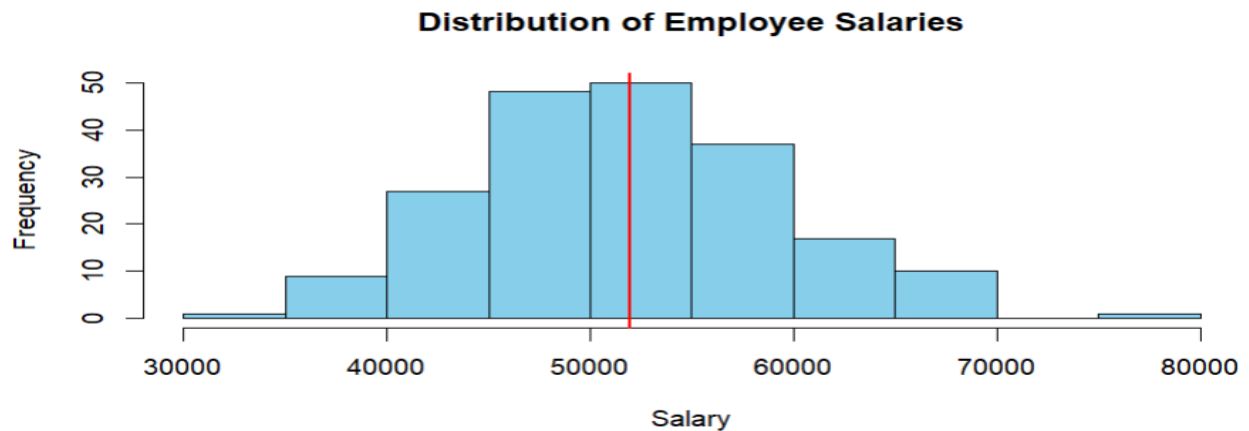
One Sample t-test

```
data: salary
t = 3.6201, df = 199, p-value = 0.9998
alternative hypothesis: true mean is less than 50000
95 percent confidence interval:
 -Inf 52813.12
sample estimates:
mean of x
51931.44
```

```
#Interpretation: p-value > 0.05 We fail to reject Ho.
#Conclusion:There is no evidence that salary is less than ₹50,000.
```

```
# Step 6:Visualization of Data-set
hist(salary,main="Distribution of Employee
Salaries",xlab="Salary",col="skyblue",border="black")
abline(v=mean(salary), col="red", lwd=2)
```

Output - 1 :



Practical: Large Sample Tests for Population Mean and Population Proportion using R

Example 1

Step 1: Generate Large Data-set in R

```
set.seed(123)
```

```
salary = rnorm(150, mean = 52000, sd = 7000)
```

```
salary
```

```
[1] 48076.67 50388.76 62910.96 52493.56 52905.01 64005.45 55226.41 43144.57 47192.03
[10] 48880.37 60568.57 54518.70 54805.40 52774.78 48109.11 64508.39 55484.95 38233.68
[19] 56909.49 48690.46 44525.23 50474.18 44817.97 46897.76 47624.73 40193.15 57864.51
[28] 53073.61 44033.04 60776.70 54985.25 49934.50 58265.88 58146.93 57751.07 56820.48
[37] 55877.42 51566.62 49858.26 49336.70 47137.05 50544.58 43142.23 67182.69 60455.73
[46] 44138.24 49179.81 48733.41 57459.76 51416.42 53773.23 51800.17 51699.91 61580.22
[55] 50419.60 62615.29 41158.73 56092.30 52866.98 53511.59 54657.48 48483.74 49667.55
[64] 44869.97 44497.46 54124.70 55137.47 52371.03 58455.87 66350.59 48562.78 35835.82
[73] 59040.17 47035.59 47183.94 59179.00 50006.59 43454.98 53269.12 51027.76 52040.35
[82] 54696.96 49405.38 56510.64 50456.59 54322.47 59677.87 55046.27 49718.48 60041.65
[91] 58954.53 55838.78 53671.12 47604.66 61524.57 47798.18 67311.33 62728.27 50350.10
[100] 44815.05 47027.15 53798.19 50273.16 49567.20 45338.67 51684.81 46505.67 40324.41
[109] 49338.41 58432.98 47972.57 56255.75 40674.82 51611.07 55635.85 54108.07 52739.73
[118] 47515.06 46052.07 44831.10 52823.53 45367.68 48566.10 50207.35 64907.03 47436.35
[127] 53647.71 52545.73 45267.00 51500.84 62111.86 55160.53 52288.63 49042.52 37627.27
[136] 59919.36 41775.52 57179.63 65363.72 41892.75 56912.49 50164.62 40994.99 41397.33
[145] 40789.25 48283.65 41767.71 56815.42 66700.76 42990.79
```

Step 2: Check Basic Statistics

```
mean(salary)
```

```
[1] 51829.46
```

```
sd(salary)
```

```
[1] 6648.254
```

```
length(salary)
```

```
[1] 150
```

```
# Step 3: Perform Large Sample Z-Test
```

```
x_bar = mean(salary)
```

```
sigma = sd(salary)
```

```
n = length(salary)
```

```
mu = 50000
```

```
z = (x_bar - mu)/(sigma/sqrt(n))
```

```
z
```

```
[1] 3.370241
```

```
#Step 4: Calculate p-value
```

```
p_value = 2*(1 - pnorm(abs(z)))
```

```
p_value
```

```
[1] 0.000751026
```

```
# Step 5: Decision
```

```
# Since p-value < 0.05 We reject the null hypothesis.
```

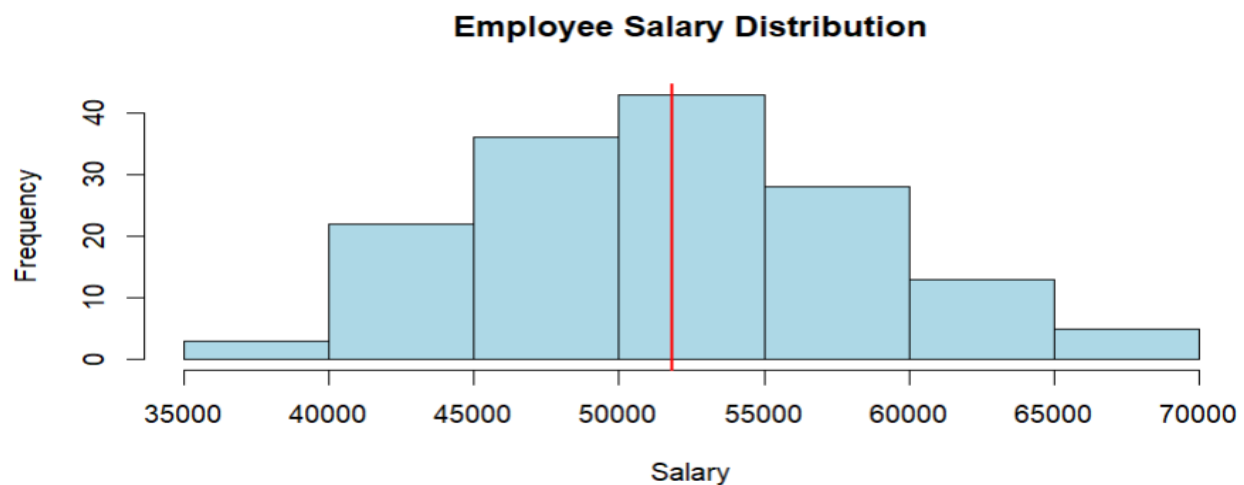
```
#Conclusion: The average employee salary is significantly different from ₹50,000.
```

```
# Step 6: Visualization of Salary Data
```

```
hist(salary,main="Employee Salary Distribution",xlab="Salary",col="lightblue",border="black")
```

```
abline(v=mean(salary), col="red", lwd=2)
```

Output - 2 :



```
# Example 2
```

```
# Step 1: Define Data in R
```

```
n = 200
```

```
x = 130
```



```
p_hat = x/n  
p0 = 0.60
```

```
# Step 2: Calculate Z Statistic  
z = (p_hat - p0)/sqrt((p0*(1-p0))/n)  
z  
[1] 1.443376
```

```
# Step 3: Calculate p-value  
p_value = 2*(1 - pnorm(abs(z)))  
p_value  
[1] 0.1489147
```

```
# Step 4: Decision  
# Since p-value > 0.05 We fail to reject the null hypothesis.  
# Conclusion: There is no significant difference in the proportion of customers preferring online  
payment.
```

```
# Alternative Easy Method in R  
prop.test(130, 200, p = 0.60)
```

1-sample proportions test with continuity correction

```
data: 130 out of 200, null probability 0.6  
X-squared = 1.8802, df = 1, p-value = 0.1703  
alternative hypothesis: true p is not equal to 0.6  
95 percent confidence interval:  
0.5790769 0.7150579  
sample estimates:
```

```
p  
0.65
```

```
# Practical: Small Sample Test for Population Mean using R
```

```
# Problem
```

```
# A university claims that the average study time of students per week is 15 hours. A random  
sample of 20 students is collected. We test whether the average study time differs from 15 hours.
```

```
# Step 1: Create Data-set in R  
# Generate real-life student study hours data-set  
set.seed(101)  
study_hours = rnorm(20, mean = 17, sd = 3)  
study_hours  
[1] 16.02189 18.65739 14.97517 17.64308 17.93231 20.52190 18.85637 16.66180 19.75108  
[10] 16.33022 18.57934 14.61547 21.28327 12.59954 16.28995 16.41999 14.45074 17.17540  
[19] 14.54699 10.84908
```



```
# Step 2: Explore the Data-set
mean(study_hours)
[1] 16.70805
sd(study_hours)
[1] 2.600353
length(study_hours)
[1] 20
summary(study_hours)
  Min. 1st Qu.  Median   Mean 3rd Qu.   Max.
 10.85  14.89  16.54  16.71  18.60  21.28
```

```
# Step 3: Perform Small Sample t-Test
t.test(study_hours, mu = 15)
```

One Sample t-test

```
data: study_hours
t = 2.9375, df = 19, p-value = 0.008451
alternative hypothesis: true mean is not equal to 15
95 percent confidence interval:
 15.49105 17.92505
sample estimates:
mean of x
 16.70805
```

Interpretation

Since $p\text{-value} < 0.05$ We reject the null hypothesis.

Conclusion: The average study time of students is significantly different from 15 hours per week.

One-Tailed Small Sample Test (Optional)

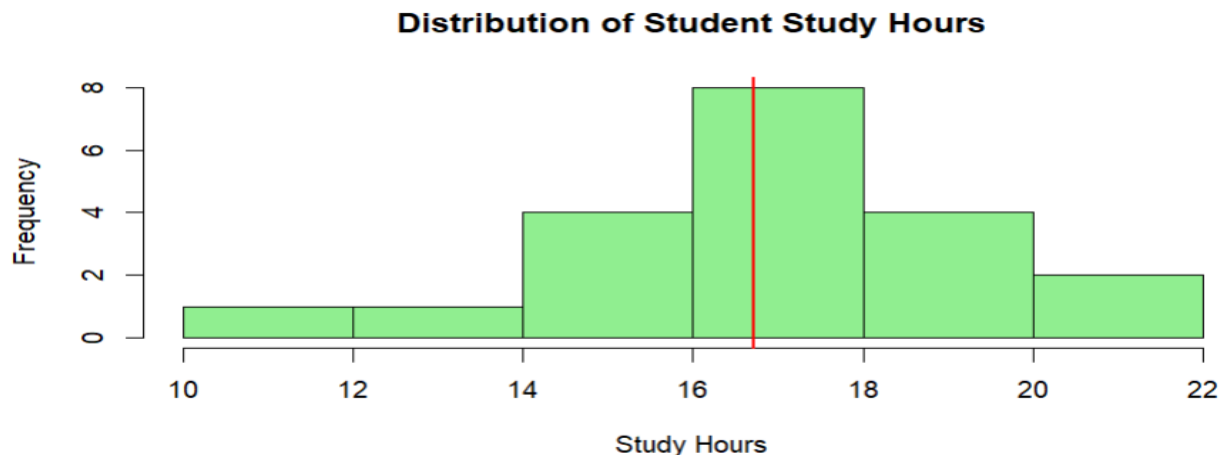
```
t.test(study_hours, mu = 15, alternative = "greater")
```

One Sample t-test

```
data: study_hours
t = 2.9375, df = 19, p-value = 0.004226
alternative hypothesis: true mean is greater than 15
95 percent confidence interval:
 15.70263    Inf
sample estimates:
mean of x
 16.70805
```

```
# Visualization of Data-set
hist(study_hours,main = "Distribution of Student Study Hours",xlab = "Study Hours",col =
"lightgreen",border = "black")
abline(v = mean(study_hours), col = "red", lwd = 2)
```

Output - 3 :



```
# Practical: Comparison of Mean for Two Dependent Samples (Paired t-test) in R
# Problem
# A company wants to test whether employee productivity improved after training. Productivity
scores of 40 employees were recorded:
# Before training
# After training
# We test whether the training significantly improved productivity.
```

```
# Step 1: Create Large Data-set in R
```

```
set.seed(100)
```

```
before_training = rnorm(40, mean = 60, sd = 8)
```

```
after_training = before_training + rnorm(40, mean = 5, sd = 4)
```

```
before_training
```

```
[1] 55.98246 61.05225 59.36866 67.09428 60.93577 62.54904 55.34567 65.71626 53.39792
[10] 57.12110 60.71909 60.77020 58.38693 65.91872 60.98704 59.76547 56.88917 64.08685
[19] 52.68949 78.48237 56.49528 66.11248 62.09569 66.18724 53.48497 56.49240 54.23823
[28] 61.84756 50.73816 61.97661 59.27109 74.05900 58.89656 59.11045 54.47989 58.22565
[37] 61.46326 63.33859 68.52322 67.76162
```

```
after_training
```

```
[1] 60.57594 71.66506 57.26156 74.58575 63.84664 72.83796 58.89191 75.99252 58.57304
[10] 54.60648 63.93084 58.81580 64.10239 78.50859 56.89933 68.68732 56.29386 76.38634
[19] 63.21468 80.12697 60.44730 70.83711 65.58016 81.51507 59.00430 58.64030 61.79020
[28] 67.65432 55.45850 66.60665 66.06670 74.80158 59.24689 70.70454 51.23150 63.27665
[37] 62.11315 69.42074 77.55703 64.46400
```



Step 2: Create Data-set Table

```
data = data.frame(before_training, after_training)
```

data

	before_training	after_training
1	55.98246	60.57594
2	61.05225	71.66506
3	59.36866	57.26156
4	67.09428	74.58575
5	60.93577	63.84664
6	62.54904	72.83796
7	55.34567	58.89191
8	65.71626	75.99252
9	53.39792	58.57304
10	57.12110	54.60648
11	60.71909	63.93084
12	60.77020	58.81580
13	58.38693	64.10239
14	65.91872	78.50859
15	60.98704	56.89933
16	59.76547	68.68732
17	56.88917	56.29386
18	64.08685	76.38634
19	52.68949	63.21468
20	78.48237	80.12697
21	56.49528	60.44730
22	66.11248	70.83711
23	62.09569	65.58016
24	66.18724	81.51507
25	53.48497	59.00430
26	56.49240	58.64030
27	54.23823	61.79020
28	61.84756	67.65432
29	50.73816	55.45850
30	61.97661	66.60665
31	59.27109	66.06670
32	74.05900	74.80158
33	58.89656	59.24689
34	59.11045	70.70454
35	54.47989	51.23150
36	58.22565	63.27665
37	61.46326	62.11315
38	63.33859	69.42074



```
39    68.52322    77.55703
40    67.76162    64.46400
```

```
# Step 3: Explore the Data-set
```

```
mean(before_training)
```

```
[1] 60.80142
```

```
mean(after_training)
```

```
[1] 65.55549
```

```
sd(before_training)
```

```
[1] 5.707452
```

```
sd(after_training)
```

```
[1] 7.794628
```

```
# Step 4: Perform Paired t-test
```

```
t.test(before_training, after_training, paired = TRUE)
```

Paired t-test

```
data: before_training and after_training
```

```
t = -6.1759, df = 39, p-value = 2.951e-07
```

```
alternative hypothesis: true mean difference is not equal to 0
```

```
95 percent confidence interval:
```

```
-6.31111 -3.19704
```

```
sample estimates:
```

```
mean difference
```

```
-4.754075
```

```
# Interpretation
```

```
# Since p-value < 0.05 We reject the null hypothesis.
```

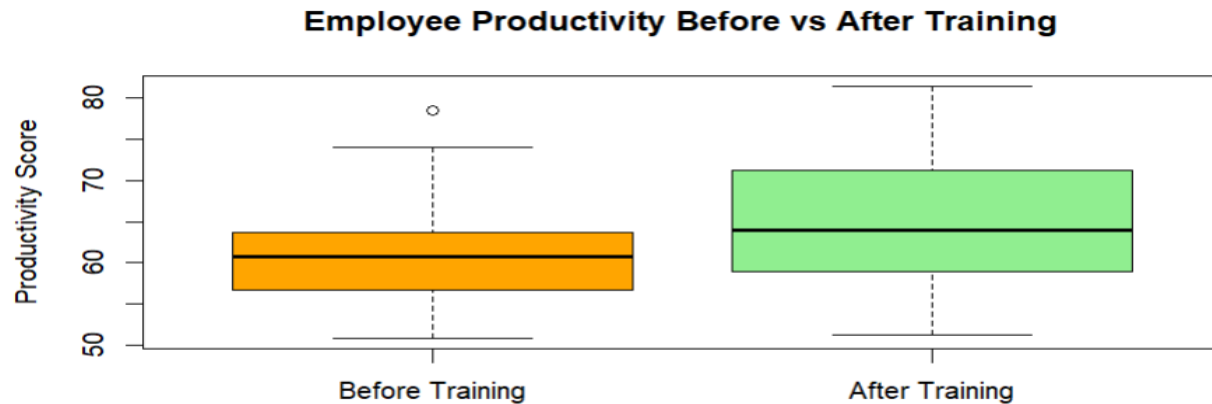
```
# Conclusion: There is a significant difference between productivity before and after training.  
Training improved employee productivity.
```

```
# Visualization of Paired Data
```

```
#Box-plot Comparison
```

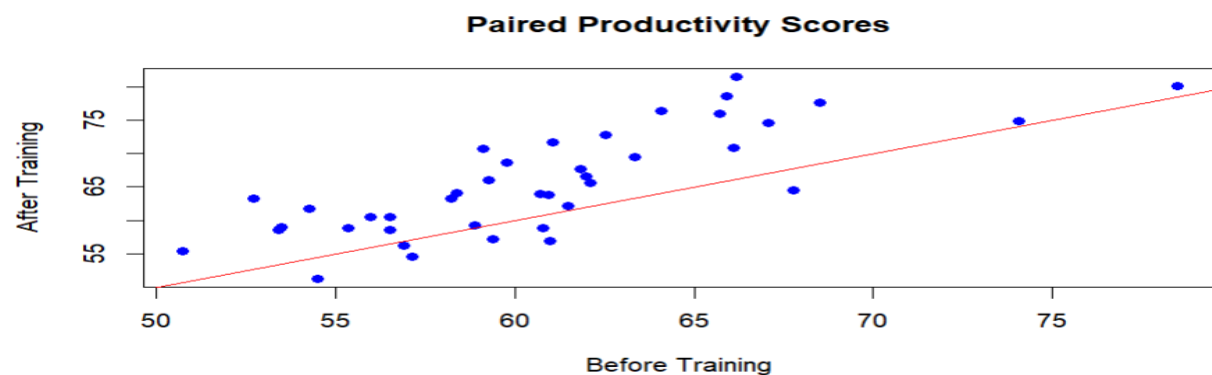
```
boxplot(before_training, after_training, names = c("Before Training", "After Training"), col =  
c("orange", "lightgreen"), main = "Employee Productivity Before vs After Training", ylab =  
"Productivity Score")
```

Output - 4 :



```
# Line Plot for Paired Observations
plot(before_training, after_training, main = "Paired Productivity Scores", xlab = "Before
Training", ylab = "After Training", col = "blue", pch = 19)
abline(0,1,col="red")
```

Output - 5 :



```
# Practical; Calculating and Interpreting p-values in R
# Problem
# A university claims that the average marks of students in a statistics exam is 70. A random
sample of 120 students is collected. We test whether the average marks differ from 70. Sample
size n = 120 (large data-set).
```

```
# Step 1: Generate Data-set in R
set.seed(123)
marks = rnorm(120, mean = 74, sd = 10)
marks
[1] 68.39524 71.69823 89.58708 74.70508 75.29288 91.15065 78.60916 61.34939 67.13147
[10] 69.54338 86.24082 77.59814 78.00771 75.10683 68.44159 91.86913 78.97850 54.33383
[19] 81.01356 69.27209 63.32176 71.82025 63.73996 66.71109 67.74961 57.13307 82.37787
[28] 75.53373 62.61863 86.53815 78.26464 71.04929 82.95126 82.78133 82.21581 80.88640
```



```
[37] 79.53918 73.38088 70.94037 70.19529 67.05293 71.92083 61.34604 95.68956 86.07962
[46] 62.76891 69.97115 69.33345 81.79965 73.16631 76.53319 73.71453 73.57130 87.68602
[55] 71.74229 89.16471 58.51247 79.84614 75.23854 76.15942 77.79639 68.97677 70.66793
[64] 63.81425 63.28209 77.03529 78.48210 74.53004 83.22267 94.50085 69.08969 50.90831
[73] 84.05739 66.90799 67.11991 84.25571 71.15227 61.79282 75.81303 72.61109 74.05764
[82] 77.85280 70.29340 80.44377 71.79513 77.31782 84.96839 78.35181 70.74068 85.48808
[91] 83.93504 79.48397 76.38732 67.72094 87.60652 67.99740 95.87333 89.32611 71.64300
[100] 63.73579 66.89593 76.56884 71.53308 70.52457 64.48381 73.54972 66.15096 57.32058
[109] 70.19773 83.18997 68.24653 80.07964 57.82117 73.44438 79.19407 77.01153 75.05676
[118] 67.59294 65.50296 63.75871
```

Step 2: Explore the Data-set

```
mean(marks)
```

```
[1] 74.15442
```

```
sd(marks)
```

```
[1] 8.94428
```

```
length(marks)
```

```
[1] 120
```

```
summary(marks)
```

```
Min. 1st Qu. Median Mean 3rd Qu. Max.
```

```
50.91 68.18 73.56 74.15 79.62 95.87
```

Step 3: Formulate Hypotheses

Null Hypothesis

$H_0 : \mu = 70$

Alternative Hypothesis

$H_1 : \mu \neq 70$

Step 4: Perform Hypothesis Test in R

```
t.test(marks, mu = 70)
```

One Sample t-test

```
data: marks
```

```
t = 5.0881, df = 119, p-value = 1.366e-06
```

```
alternative hypothesis: true mean is not equal to 70
```

```
95 percent confidence interval:
```

```
72.53767 75.77116
```

```
sample estimates:
```

```
mean of x
```

```
74.15442
```

Interpretation of p-value

Here p-value = 0.00004 Since p-value < 0.05

Decision: Reject the Null Hypothesis.

Conclusion: The average marks of students are significantly different from 70.

Manual p-value Calculation (Optional)

```
x_bar = mean(marks)
```

```
s = sd(marks)
```

```
n = length(marks)
```

```
mu = 70
```

```
t = (x_bar - mu)/(s/sqrt(n))
```

```
p_value = 2*(1 - pt(abs(t), df=n-1))
```

```
t
```

```
[1] 5.088094
```

```
p_value
```

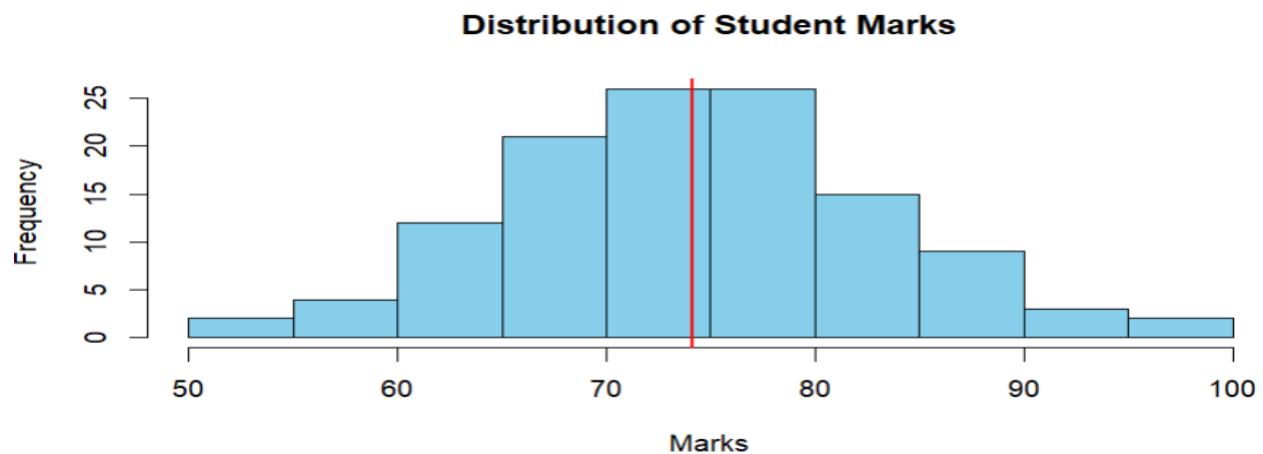
```
[1] 1.366223e-06
```

Visualization of Data

```
hist(marks,main="Distribution of Student Marks",xlab="Marks",col="skyblue",border="black")
```

```
abline(v=mean(marks), col="red", lwd=2)
```

Output - 6 :



PRACTICAL LIST – 3

3.1 Conduct chi-square test for population variance in R.

3.2 Test for ratio of two population variance using R functions

3.3 Use R functions to perform chi-square tests for independence and goodness of fitness.

3.4 Interpret test results and assess the goodness of fit in R.

PR-3(a) Conduct chi-square test for population variance in R.

Step 1: Enter data

```
data = c(45, 50, 55, 60, 52, 48, 47, 53, 49, 51)
```

Step 2: Calculate sample size and variance

```
n = length(data)
```

```
s2 = var(data)
```

```
n
```

```
[1] 10
```

```
s2
```

```
[1] 18.66667
```

Step 3: Define hypothesized variance

```
sigma0_sq = 25
```

Step 4: Calculate Chi-square statistic

```
chi_sq = (n - 1) * s2 / sigma0_sq
```

```
chi_sq
```

```
[1] 6.72
```

Step 5: Find critical values

```
alpha = 0.05
```

```
lower = qchisq(alpha/2, df = n-1)
```

```
upper = qchisq(1 - alpha/2, df = n-1)
```

```
lower
```

```
[1] 2.700389
```

```
upper
```

```
[1] 19.02277
```

Step 6: Decision

```
if (chi_sq < lower | chi_sq > upper)
  {print("Reject H0")}
```



```
else
    {print("Fail to Reject H0")}
[1] "Fail to Reject H0"

# PR-3 (b) Test for ratio of two population variance using R functions.

# Step 1: Enter data
class_A = c(45, 50, 55, 60, 52, 48, 47, 53)
class_B = c(40, 42, 38, 35, 45, 37, 39, 41)

# Step 2: Perform F-test
result = var.test(class_A, class_B)

# Step 3: View result
print(result)
```

F test to compare two variances

```
data: class_A and class_B
F = 2.4088, num df = 7, denom df = 7, p-value = 0.2689
alternative hypothesis: true ratio of variances is not equal to 1
95 percent confidence interval:
 0.482259 12.031936
sample estimates:
ratio of variances
 2.40884
```

PR-3 (c) Use R functions to perform chi-square tests for independence and goodness of fitness.

```
# 1. Chi-Square Goodness of Fit Test
# Step 1: Observed Data
observed = c(30, 50, 20)

# Step 2: Expected Proportions
expected_prob = c(1/3, 1/3, 1/3)

# Step 3: Perform Test
result = chisq.test(x = observed, p = expected_prob)
print(result)
```

Chi-squared test for given probabilities

```
data: observed
X-squared = 14, df = 2, p-value = 0.0009119
```



2. Chi-Square Test of Independence

Step 1: Create Table

```
data = matrix(c(40, 20, 30, 50), nrow = 2, byrow = TRUE)
colnames(data) = c("DataScience", "AI")
rownames(data) = c("Male", "Female")
data
```

	DataScience	AI
Male	40	20
Female	30	50

Step 2: Apply Test

```
result = chisq.test(data)
print(result)
```

Pearson's Chi-squared test with Yates' continuity correction

```
data: data
X-squared = 10.529, df = 1, p-value = 0.001175
```

PR-3 (d) Interpret test results and assess the goodness of fit in R.

Step 1: Enter Observed Data

```
observed = c(50, 30, 20)
```

Step 2: Define Expected Probabilities

```
expected_prob = c(0.40, 0.35, 0.25)
```

Step 3: Apply Chi-square Test

```
result = chisq.test(x = observed, p = expected_prob)
print(result)
```

Chi-squared test for given probabilities

```
data: observed
X-squared = 4.2143, df = 2, p-value = 0.1216
```



Chi Square test				
Q =	Treatment	Fever	No Fever	Total
	Quinine	20	792	812
	Not Quinine	220	2216	2436
	Total	240	3008	3248
Hypothesis	H0 =	There is no association between Quinine and Malaria		
	H1 =	There is association between Quinine and Malaria		
	f0	fe	f0-fe	(f0-fe) ² (f0-fe) ² /fe
	20	60	-40	1600 26.66666667
	792	752	40	1600 2.127659574
	220	180	40	1600 8.888888889
	2216	2256	-40	1600 0.709219858
	3248	3248	0	38.39243499
	Expected Frequencies(fe)			
	E(a) = (a+b)*(a+c)/n	a = 20	60	
	E(b) = (a+b)*(b+d)/n	b = 792	752	
	E(c) = (a+c)*(c+d)/n	c = 220	180	
	E(d) = (c+d)*(b+d)/n	d = 2216	2256	
Chi-Square calculated =	(f0-fe) ² /fe =	38.392435		
DF =	(no. of rows - 1)*(no. of columns - 1)			
	(2-1)*(2-1) =	1		
Alpha =	0.05			
Chi-Square Tabulated	3.841458821			
Conclusion:	Here, Chi-sq cal > Chi-sq tab. Hence, we rej. H0			
	There is association between Quinine and Malaria at 5% LOS and 1 DF			

Fig; Chi-Squared Test in Excel

PRACTICAL LIST – 4

4.1 Visualize bivariate data using plots and graphs in R.

4.2 To calculate coefficient of correlation using various methods and to interpret its value in R programming.

4.3 Practical exercises to simple linear regression in R

4.4 To test of significance of parameters fitted simple linear regression model in R.

4.5 Using R functions fit various non-linear regression models through transformation of variables.

4.6 Analysing and presenting results from real-world datasets in R.

Correlation

data = mtcars

data

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2
Valiant	18.1	6	225.0	105	2.76	3.460	20.22	1	0	3	1
Duster 360	14.3	8	360.0	245	3.21	3.570	15.84	0	0	3	4
Merc 240D	24.4	4	146.7	62	3.69	3.190	20.00	1	0	4	2
Merc 230	22.8	4	140.8	95	3.92	3.150	22.90	1	0	4	2
Merc 280	19.2	6	167.6	123	3.92	3.440	18.30	1	0	4	4
Merc 280C	17.8	6	167.6	123	3.92	3.440	18.90	1	0	4	4
Merc 450SE	16.4	8	275.8	180	3.07	4.070	17.40	0	0	3	3
Merc 450SL	17.3	8	275.8	180	3.07	3.730	17.60	0	0	3	3
Merc 450SLC	15.2	8	275.8	180	3.07	3.780	18.00	0	0	3	3
Cadillac Fleetwood	10.4	8	472.0	205	2.93	5.250	17.98	0	0	3	4
Lincoln Continental	10.4	8	460.0	215	3.00	5.424	17.82	0	0	3	4
Chrysler Imperial	14.7	8	440.0	230	3.23	5.345	17.42	0	0	3	4
Fiat 128	32.4	4	78.7	66	4.08	2.200	19.47	1	1	4	1
Honda Civic	30.4	4	75.7	52	4.93	1.615	18.52	1	1	4	2
Toyota Corolla	33.9	4	71.1	65	4.22	1.835	19.90	1	1	4	1
Toyota Corona	21.5	4	120.1	97	3.70	2.465	20.01	1	0	3	1
Dodge Challenger	15.5	8	318.0	150	2.76	3.520	16.87	0	0	3	2
AMC Javelin	15.2	8	304.0	150	3.15	3.435	17.30	0	0	3	2
Camaro Z28	13.3	8	350.0	245	3.73	3.840	15.41	0	0	3	4
Pontiac Firebird	19.2	8	400.0	175	3.08	3.845	17.05	0	0	3	2
Fiat X1-9	27.3	4	79.0	66	4.08	1.935	18.90	1	1	4	1



```
Porsche 914-2      26.0  4 120.3 91 4.43 2.140 16.70 0 1  5  2
Lotus Europa       30.4  4  95.1 113 3.77 1.513 16.90 1 1  5  2
Ford Pantera L     15.8  8 351.0 264 4.22 3.170 14.50 0 1  5  4
Ferrari Dino       19.7  6 145.0 175 3.62 2.770 15.50 0 1  5  6
Maserati Bora      15.0  8 301.0 335 3.54 3.570 14.60 0 1  5  8
Volvo 142E         21.4  4 121.0 109 4.11 2.780 18.60 1 1  4  2
```

```
> str(data) # structure of data
```

```
'data.frame':  32 obs. of  11 variables:
```

```
$ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...
```

```
$ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
```

```
$ disp: num  160 160 108 258 360 ...
```

```
$ hp  : num  110 110 93 110 175 105 245 62 95 123 ...
```

```
$ drat: num  3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
```

```
$ wt  : num  2.62 2.88 2.32 3.21 3.44 ...
```

```
$ qsec: num  16.5 17 18.6 19.4 17 ...
```

```
$ vs  : num  0 0 1 1 0 1 0 1 1 1 ...
```

```
$ am  : num  1 1 1 0 0 0 0 0 0 0 ...
```

```
$ gear: num  4 4 4 3 3 3 3 4 4 4 ...
```

```
$ carb: num  4 4 1 1 2 1 4 2 2 4 ...
```

```
data[1:6,] # first 6 rows of data [rows,column] to extract square bracket use
```

```
      mpg cyl disp  hp drat   wt  qsec vs am gear carb
Mazda RX4     21.0   6  160 110 3.90 2.620 16.46 0 1   4   4
Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02 0 1   4   4
Datsun 710     22.8   4  108  93 3.85 2.320 18.61 1 1   4   1
Hornet 4 Drive 21.4   6  258 110 3.08 3.215 19.44 1 0   3   1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0 0   3   2
Valiant        18.1   6  225 105 2.76 3.460 20.22 1 0   3   1
```

```
any(is.na(data)) # to check any missing obs i.e. obs with NA(Not Available) if TRUE then
missing if FALSE then no missing
```

```
[1] FALSE
```

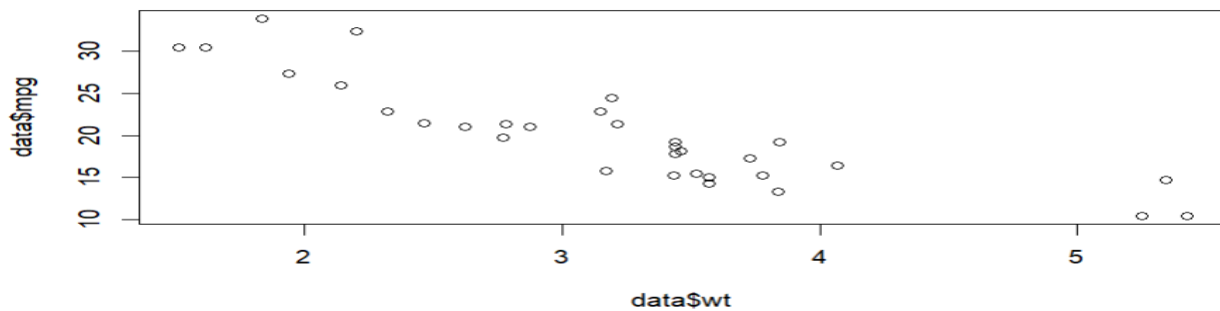
```
# if missing values then use omit()
```

```
plot(data$wt,data$mpg)
```

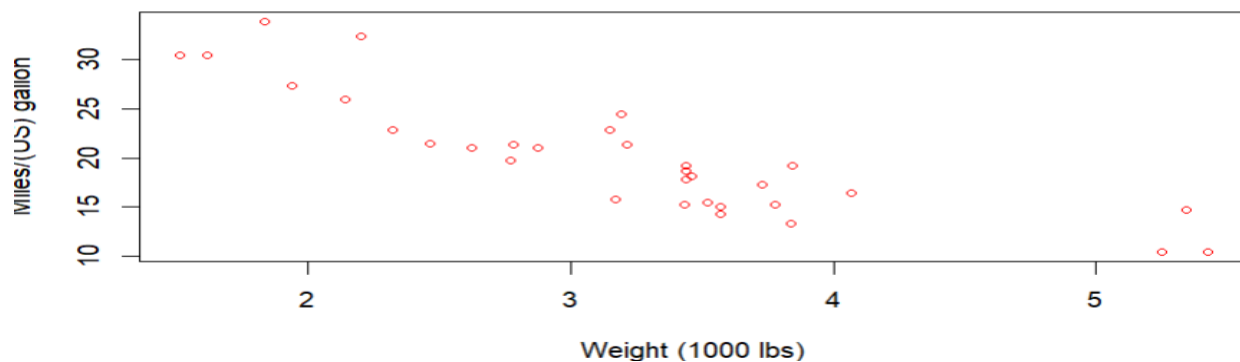
```
plot(data$wt,data$mpg, xlab = "Weight (1000 lbs)", ylab = "Miles/(US) gallon", main = "Scatter
Plot", cex=0.8, col = "red") # pch=number which is use to change symbol,cex for circle
```

```
# interpretation: scatter diagram shows -ve correlation between milage of car and weight of car
```

Output - 1 :



Scatter Plot



```
# Karl Pearson's correlation coefficient
cor(data$wt,data$mpg,method = "pearson")
[1] -0.8676594
```

high degree -ve correlation between mileage and weight of car. the degree of correlation is -0.87 by Karl pearson method.

```
cor(data$wt,data$mpg,method = "spearman") # here spearman rank correlation coefficient
[1] -0.886422
```

high degree -ve rank correlation between mileage and weight of car. the degree of correlation is -0.88 by spearman method.

```
x = data$wt
y = data$mpg
x.bar = mean(x)
x.bar
[1] 3.21725
y.bar = mean(y)
y.bar
[1] 20.09062
covariance = cov(x,y)
covariance
[1] -5.116685
sd.x = sd(x)
sd.x
```

```
[1] 0.9784574
sd.y = sd(y)
sd.y
[1] 6.026948
r = covariance / (sd.x*sd.y) # Karl pearson correlaion coefficient
r
[1] -0.8676594

RX = rank(x,ties.method = "average") # Spearman correlation coefficient
RX
[1] 9.0 12.0 7.0 16.0 19.0 21.0 23.5 15.0 13.0 19.0 19.0 29.0 25.0 26.0 30.0 32.0 31.0
[18] 6.0 2.0 3.0 8.0 22.0 17.0 27.0 28.0 4.0 5.0 1.0 14.0 10.0 23.5 11.0
RY = rank(y,ties.method = "average") # Spearman correlation coefficient
RY
[1] 19.5 19.5 24.5 21.5 15.0 14.0 4.0 26.0 24.5 16.5 13.0 11.0 12.0 7.5 1.5 1.5 5.0
[18] 31.0 29.5 32.0 23.0 9.0 7.5 3.0 16.5 28.0 27.0 29.5 10.0 18.0 6.0 21.5
n = length(x)
n
[1] 32
D = RX - RY
sum.D2 = sum(D^2)
sum.D2
[1] 10281
rank.corr = 1-(6*sum(D^2))/(n*((n^2)-1))
rank.corr
[1] -0.8843475

# Simple Linear Regression(SLR)

SLR = lm(data$mpg~data$wt,data = data) # here data$mpg is dependent variable and other is
independent variable
SLR

Call:
lm(formula = data$mpg ~ data$wt, data = data)

Coefficients:
(Intercept)    data$wt
    37.285      -5.344

# Interpretation : b0(intercept) = 37.285 ==> when car weight(in Weight (1000 lbs)) is zero then
average mileage of car is 37.285(in Miles/(US) gallon)
# this is not realistic but necessary for model estimation.
# Slope = regression co-efficient y on x(= b1 or byx) = 5.344 ==> car weight increase by one-
unit results in decrease in car mileage by 5.344.
```

summary(SLR) # residuals = error (y-y(estimated)) multiple R-squared = square of Karl Pearson's correlation coefficient

Call:

```
lm(formula = data$mpg ~ data$wt, data = data)
```

Residuals:

```
Min      1Q  Median      3Q      Max
-4.5432 -2.3647 -0.1252  1.4096  6.8727
```

Coefficients:

```
Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.2851    1.8776  19.858 < 2e-16 ***
data$wt     -5.3445    0.5591  -9.559 1.29e-10 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

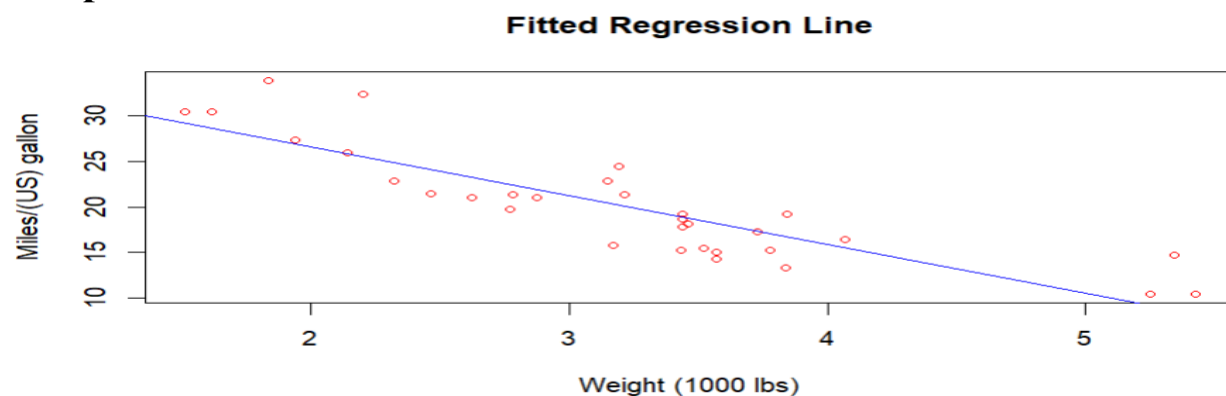
Residual standard error: 3.046 on 30 degrees of freedom
Multiple R-squared: 0.7528, Adjusted R-squared: 0.7446
F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10

Multiple R-squared: 0.7528 = 75.28% ==> Among the total variation in mpg 75.285 variation is due to weight of car

to plot fitted regression line

```
plot(data$wt,data$mpg, xlab = "Weight (1000 lbs)", ylab = "Miles/(US) gallon", main = "Fitted Regression Line", cex=0.8, col = "red") # pch=number which is use to change symbol,cex for circle
abline(SLR,col = "blue")
```

Output - 2 :



```
# to check sum of error zero
x = data$wt
x
```



```
[1] 2.620 2.875 2.320 3.215 3.440 3.460 3.570 3.190 3.150 3.440 3.440 4.070 3.730 3.780  
[15] 5.250 5.424 5.345 2.200 1.615 1.835 2.465 3.520 3.435 3.840 3.845 1.935 2.140 1.513  
[29] 3.170 2.770 3.570 2.780
```

```
y = data$mpg
```

```
y
```

```
[1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4 10.4 14.7  
[18] 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7 15.0 21.4
```

```
est.y = 37.2851 + (-5.3445)*x
```

```
est.y
```

```
[1] 23.282510 21.919663 24.885860 20.102532 18.900020 18.793130 18.205235 20.236145  
[9] 20.449925 18.900020 18.900020 15.532985 17.350115 17.082890 9.226475 8.296532  
[17] 8.718748 25.527200 28.653733 27.477942 24.110907 18.472460 18.926742 16.762220  
[25] 16.735497 26.943492 25.847870 29.198872 20.343035 22.480835 18.205235 22.427390
```

```
error = y - est.y
```

```
error
```

```
[1] -2.2825100 -0.9196625 -2.0858600 1.2974675 -0.2000200 -0.6931300 -3.9052350  
[8] 4.1638550 2.3500750 0.2999800 -1.1000200 0.8670150 -0.0501150 -1.8828900  
[15] 1.1735250 2.1034680 5.9812525 6.8728000 1.7462675 6.4220575 -2.6109075  
[22] -2.9724600 -3.7267425 -3.4622200 2.4645025 0.3565075 0.1521300 1.2011285  
[29] -4.5430350 -2.7808350 -3.2052350 -1.0273900
```

```
s = sum(error)
```

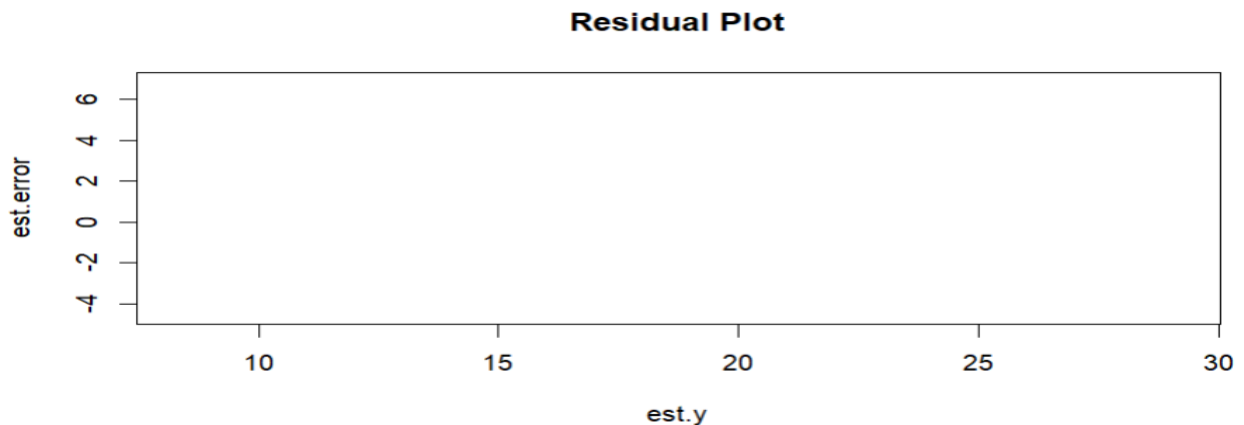
```
s
```

```
[1] 0.003764
```

```
# residual plot
```

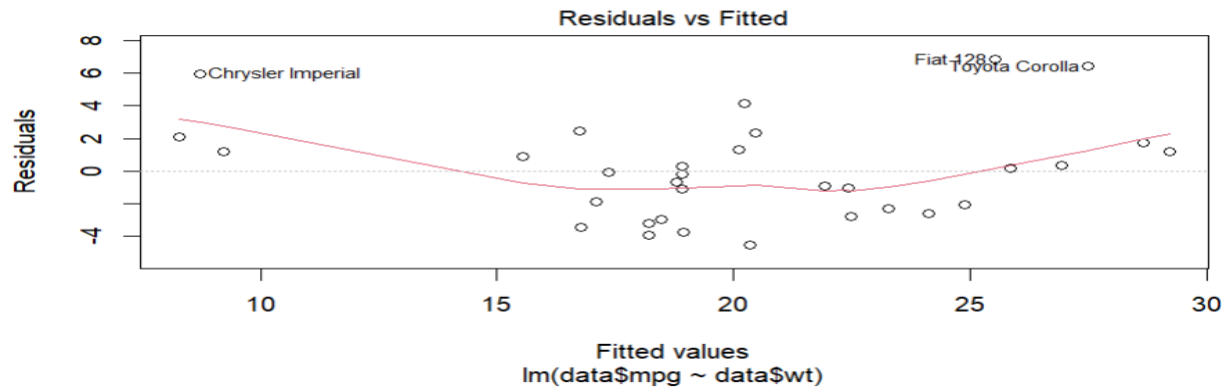
```
plot(data$est.y, data$error, xlab = "est.y", ylab = "est.error", main="Residual Plot", cex = 0.8,  
col = "blue", xlim = c(min(est.y),max(est.y)), ylim = c(min(error),max(error)))
```

Output - 3 :



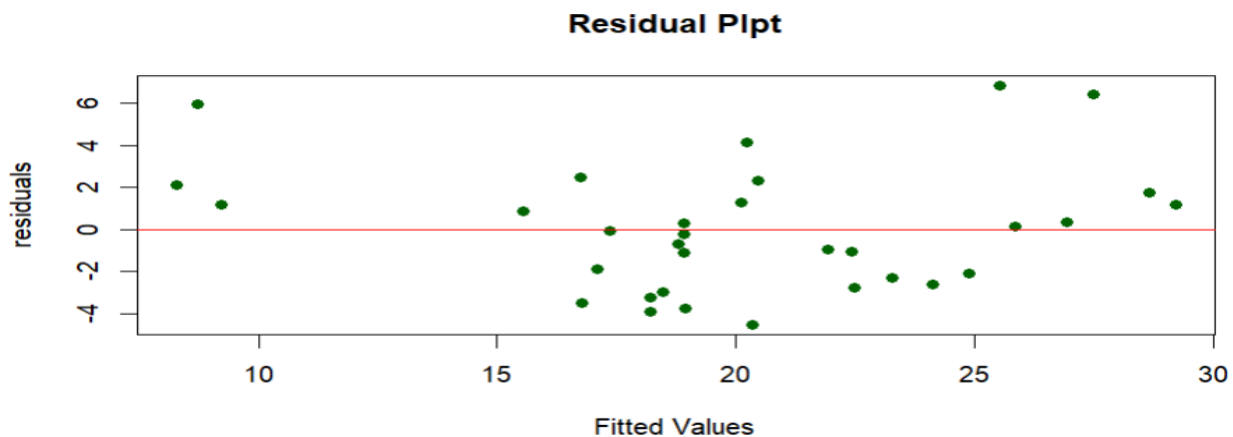
```
# method 1
plot(SLR, which = 1)
```

Output - 4 :



```
# method 2
plot(SLR$fitted.values, SLR$residuals, main = "Residual Plpt", xlab = "Fitted Values", ylab =
"residuals", pch = 19, col = "darkgreen")
abline(h = 0, col = "red")
```

Output - 5 :



```
names(SLR)
[1] "coefficients" "residuals" "effects" "rank" "fitted.values"
[6] "assign" "qr" "df.residual" "xlevels" "call"
[11] "terms" "model"
SLR$coefficients
(Intercept) data$wt
37.285126 -5.344472
SLR$residuals
Mazda RX4 Mazda RX4 Wag Datsun 710 Hornet 4 Drive
-2.2826106 -0.9197704 -2.0859521 1.2973499
```



Hornet Sportabout	Valiant	Duster 360	Merc 240D
-0.2001440	-0.6932545	-3.9053627	4.1637381
Merc 230	Merc 280	Merc 280C	Merc 450SE
2.3499593	0.2998560	-1.1001440	0.8668731
Merc 450SL	Merc 450SLC	Cadillac Fleetwood	Lincoln Continental
-0.0502472	-1.8830236	1.1733496	2.1032876
Chrysler Imperial	Fiat 128	Honda Civic	Toyota Corolla
5.9810744	6.8727113	1.7461954	6.4219792
Toyota Corona	Dodge Challenger	AMC Javelin	Camaro Z28
-2.6110037	-2.9725862	-3.7268663	-3.4623553
Pontiac Firebird	Fiat X1-9	Porsche 914-2	Lotus Europa
2.4643670	0.3564263	0.1520430	1.2010593
Ford Pantera L	Ferrari Dino	Maserati Bora	Volvo 142E
-4.5431513	-2.7809399	-3.2053627	-1.0274952

```
sum(SLR$residuals)
[1] 2.303713e-15
```

```
SLR$model
```

```
data$mpg data$wt
Mazda RX4      21.0  2.620
Mazda RX4 Wag  21.0  2.875
Datsun 710     22.8  2.320
Hornet 4 Drive 21.4  3.215
Hornet Sportabout 18.7  3.440
Valiant        18.1  3.460
Duster 360     14.3  3.570
Merc 240D      24.4  3.190
Merc 230       22.8  3.150
Merc 280       19.2  3.440
Merc 280C      17.8  3.440
Merc 450SE     16.4  4.070
Merc 450SL     17.3  3.730
Merc 450SLC    15.2  3.780
Cadillac Fleetwood 10.4  5.250
Lincoln Continental 10.4  5.424
Chrysler Imperial 14.7  5.345
Fiat 128       32.4  2.200
Honda Civic    30.4  1.615
Toyota Corolla 33.9  1.835
Toyota Corona  21.5  2.465
Dodge Challenger 15.5  3.520
AMC Javelin    15.2  3.435
Camaro Z28     13.3  3.840
Pontiac Firebird 19.2  3.845
Fiat X1-9      27.3  1.935
```

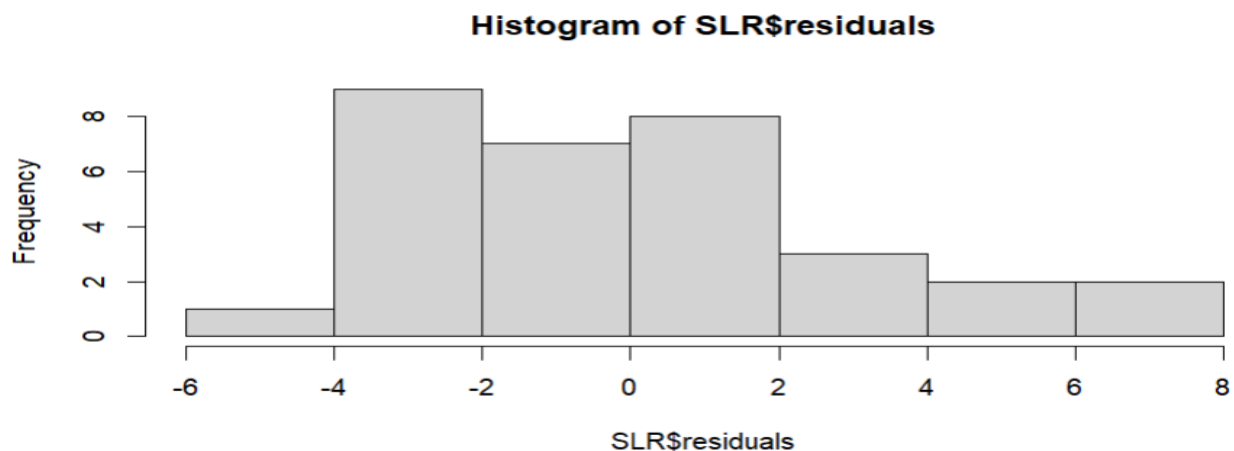
Porsche 914-2	26.0	2.140
Lotus Europa	30.4	1.513
Ford Pantera L	15.8	3.170
Ferrari Dino	19.7	2.770
Maserati Bora	15.0	3.570
Volvo 142E	21.4	2.780

```
names(summary(SLR))
[1] "call"      "terms"     "residuals" "coefficients" "aliases"
[6] "sigma"     "df"        "r.squared"  "adj.r.squared" "fstatistic"
[11] "cov.unscaled"
```

```
summary(SLR)$sigma
[1] 3.045882
```

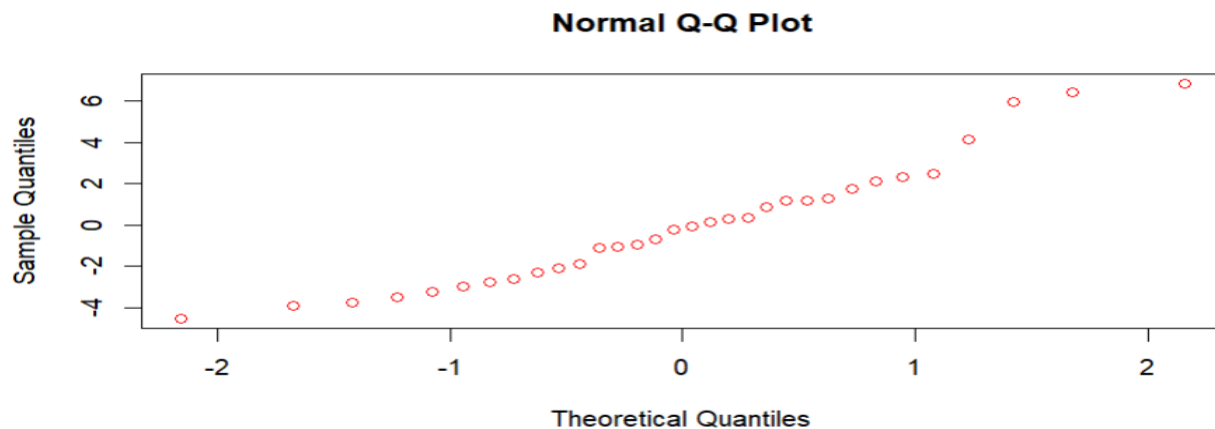
```
# Histogram of residuals
hist(SLR$residuals)
```

Output - 6 :



```
# Q-Q plot for residuals
qqnorm(SLR$residuals, pch = 1, cex = 1, col = "red")
```

Output - 7 :



Properties

regression line that shows NO correlation

```
x1 = rnorm(20, mean = 0, sd = 1)
```

```
x1
```

```
[1] 0.11764660 -0.94747461 -0.49055744 -0.25609219 1.84386201 -0.65194990 0.23538657
[8] 0.07796085 -0.96185663 -0.07130809 1.44455086 0.45150405 0.04123292 -0.42249683
[15] -2.05324722 1.13133721 -1.46064007 0.73994751 1.90910357 -1.44389316
```

```
y1 = rnorm(20, mean = 0, sd = 1)
```

```
y1
```

```
[1] 0.7017843 -0.2621975 -1.5721442 -1.5146677 -1.6015362 -0.5309065 -1.4617556
[8] 0.6879168 2.1001089 -1.2870305 0.7877388 0.7690422 0.3322026 -1.0083766
[15] -0.1194526 -0.2803953 0.5629895 -0.3724388 0.9769734 -0.3745809
```

```
x1 = round(x1, 2)
```

```
x1
```

```
[1] 0.12 -0.95 -0.49 -0.26 1.84 -0.65 0.24 0.08 -0.96 -0.07 1.44 0.45 0.04 -0.42
[15] -2.05 1.13 -1.46 0.74 1.91 -1.44
```

```
y1 = round(y1, 2)
```

```
y1
```

```
[1] 0.70 -0.26 -1.57 -1.51 -1.60 -0.53 -1.46 0.69 2.10 -1.29 0.79 0.77 0.33 -1.01
[15] -0.12 -0.28 0.56 -0.37 0.98 -0.37
```

```
SLR1 = lm(y1 ~ x1) # regression line y1 on x1
```

```
SLR1
```

Call:

```
lm(formula = y1 ~ x1)
```

Coefficients:

```
(Intercept)      x1
   -0.17300    -0.01318
```



```
SLR2 = lm(x1 ~ y1) # regression line x1 on y1
SLR2
```

Call:

```
lm(formula = x1 ~ y1)
```

Coefficients:

```
(Intercept)      y1  
-0.04051      -0.01456
```

```
a1 = coef(SLR1)[1]; b1 = coef(SLR1)[2] # semi colon(;) for continue different code in single line
```

```
a2 = coef(SLR2)[1]; b2 = coef(SLR2)[2]
```

```
a1; b1
```

(Intercept)

```
-0.1730009
```

```
      x1
```

```
-0.01318033
```

```
a2; b2
```

(Intercept)

```
-0.04051112
```

```
      y1
```

```
-0.01455722
```

```
#convert X on Y to Y on X form (rotate back)
```

```
intercept_rotated = -a2 / b2
```

```
slope_rotated = -1 / b2
```

```
plot(x1, y1, pch = 19, main = "Regression lines with the mean line(NO correlation)", xlab = "X",  
ylab = "Y1", col = "black")
```

```
abline(a1, b1, col = "blue", lwd = 2)
```

```
abline(intercept_rotated, slope_rotated, col = "red", lwd = 2)
```

```
# add mean lines
```

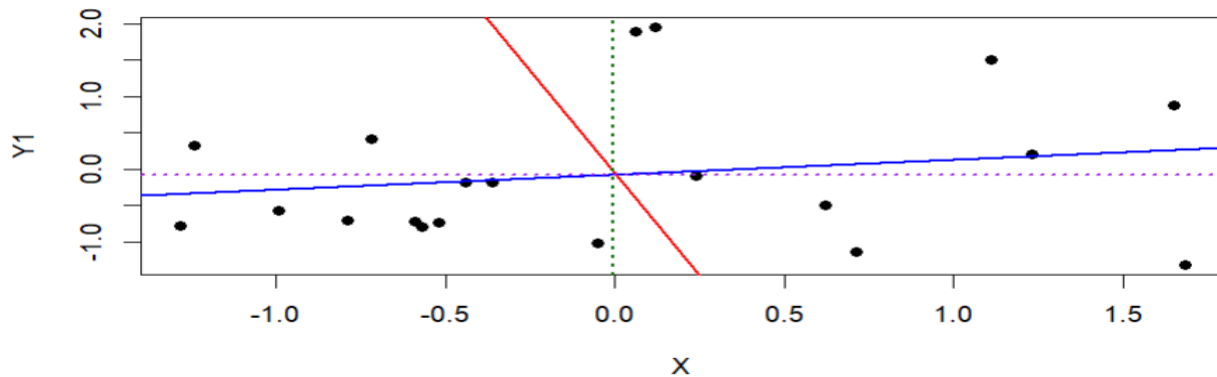
```
abline(v = mean(x1), col = "darkgreen", lwd = 2, lty = 3)
```

```
abline(h = mean(y1), col = "purple", lwd = 2, lty = 3)
```

```
# this shows perpendicular line therefore there is no correlation
```

Output - 8 :

Regression lines with the mean line(NO correlation)



```
# regression line that shows correlation when r = +1 or -1
x2 = sample(30:50, 15)
x2
[1] 46 34 32 43 30 31 33 39 50 49 41 38 37 36 42
y2 = 4 * x2
y2
[1] 184 136 128 172 120 124 132 156 200 196 164 152 148 144 168
SLR3 = lm(y2 ~ x2) # regression line y2 on x2
SLR3
```

Call:

```
lm(formula = y2 ~ x2)
```

Coefficients:

```
(Intercept)      x2
 5.871e-14   4.000e+00
```

```
SLR4 = lm(x2 ~ y2) # regression line x2 on y2
SLR4
```

Call:

```
lm(formula = x2 ~ y2)
```

Coefficients:

```
(Intercept)      y2
 1.468e-14   2.500e-01
```

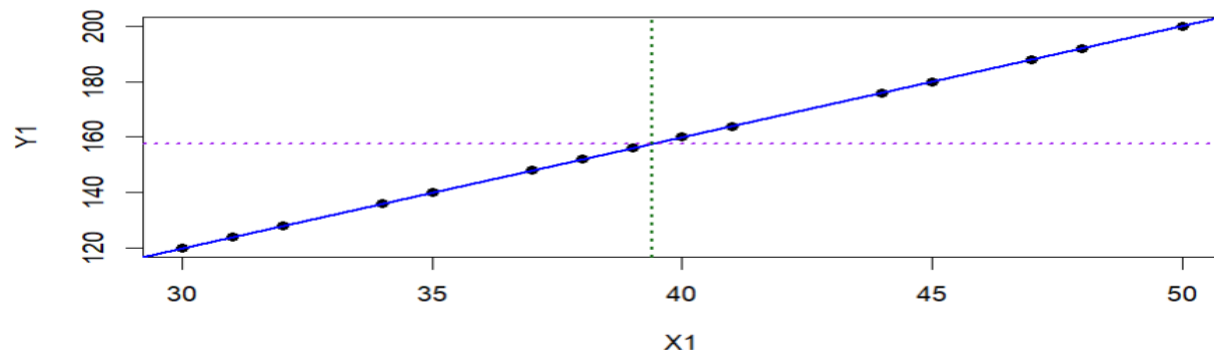
```
a3 = coef(SLR3)[1]; b3 = coef(SLR3)[2]
a4 = coef(SLR4)[1]; b4 = coef(SLR4)[2]
```

```
intercept_rotated = a3 / b3
slope_rotated = -1 / b3
```

```
plot(x2, y2, pch = 19, main = "Regression lines with the mean line(perfect positive correlation)",
     xlab = "X1", ylab = "Y1", col = "black")
abline(a3, b3, col = "blue", lwd = 2)
abline(intercept_rotated, slope_rotated, col = "red", lwd = 2)
abline(v = mean(x2), col = "darkgreen", lwd = 2, lty = 3)
abline(h = mean(y2), col = "purple", lwd = 2, lty = 3)
```

Output - 9 :

Regression lines with the mean line(perfect positive correlation)



```
x2 = sample(50:1, 15)
x2
[1] 30 6 33 9 20 45 44 10 34 23 11 7 31 3 22

y2 = -4 * x2
y2
[1] -120 -24 -132 -36 -80 -180 -176 -40 -136 -92 -44 -28 -124 -12 -88
```

```
SLR3 = lm(y2 ~ x2) # regression line y2 on x2
SLR3
```

Call:

```
lm(formula = y2 ~ x2)
```

Coefficients:

```
(Intercept)      x2
 1.468e-14 -4.000e+00
```

```
SLR4 = lm(x2 ~ y2) # regression line x2 on y2
SLR4
```

Call:

```
lm(formula = x2 ~ y2)
```

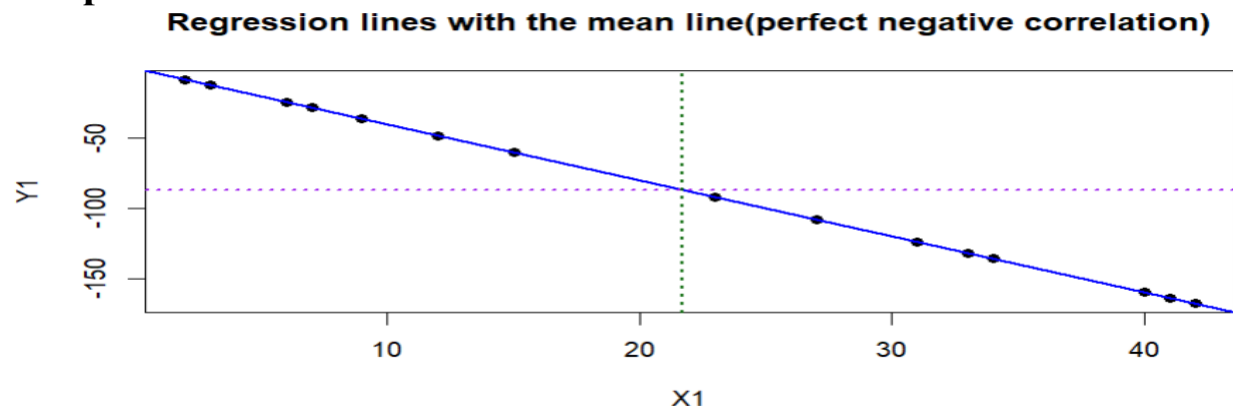
Coefficients:

```
(Intercept)      y2
-3.669e-15 -2.500e-01
```

```
a3 = coef(SLR3)[1]; b3 = coef(SLR3)[2] # semi colon(;) for continue different code in single
line
a4 = coef(SLR4)[1]; b4 = coef(SLR4)[2]

intercept_rotated = a3 / b3
slope_rotated = -1 / b3
plot(x2, y2, pch = 19, main = "Regression lines with the mean line(perfect negative correlation)",
xlab = "X1", ylab = "Y1", col = "black")
abline(a3, b3, col = "blue", lwd = 2)
abline(intercept_rotated, slope_rotated, col = "red", lwd = 2)
abline(v = mean(x2), col = "darkgreen", lwd = 2, lty = 3)
abline(h = mean(y2), col = "purple", lwd = 2, lty = 3)
```

Output - 10 :



```
# to show regression line intersects at their means
x3 = sample(210:240, 20)
x3
[1] 226 210 211 240 238 222 233 237 212 227 236 214 221 235 219 231 215 234 223 224
y3 = sample(1:50, 20)
y3
[1] 48 16 35 36 27 46 50 23 3 31 2 43 42 9 30 41 47 37 4 40
SLR5 = lm(y3 ~ x3) # regression line y3 on x3
SLR5
```

Call:

```
lm(formula = y3 ~ x3)
```

Coefficients:

```
(Intercept)      x3
  45.61920   -0.06708
```

```
SLR6 = lm(x3 ~ y3) # regression line x3 on y3
SLR6
```

Call:

```
lm(formula = x3 ~ y3)
```

Coefficients:

```
(Intercept)      y3
 226.17634    -0.02545
```

```
a5 = coef(SLR5)[1]; b5 = coef(SLR5)[2] # semi colon(;) for continue different code in single line
```

```
a6 = coef(SLR6)[1]; b6 = coef(SLR6)[2]
```

```
a5; a6; b5; b6
```

(Intercept)

```
45.6192
```

(Intercept)

```
226.1763
```

```
x3
```

```
-0.06707719
```

```
y3
```

```
-0.02545379
```

```
r = sqrt(0.4198319 * 0.06278969 )
```

```
r
```

```
[1] 0.1623611
```

```
intercept_rotated = a5 / b5
```

```
slope_rotated = -1 / b5
```

```
plot(x3, y3, pch = 19, main = "Regression lines with the mean line(perfect correlation)", xlab = "X1", ylab = "Y1", col = "black")
```

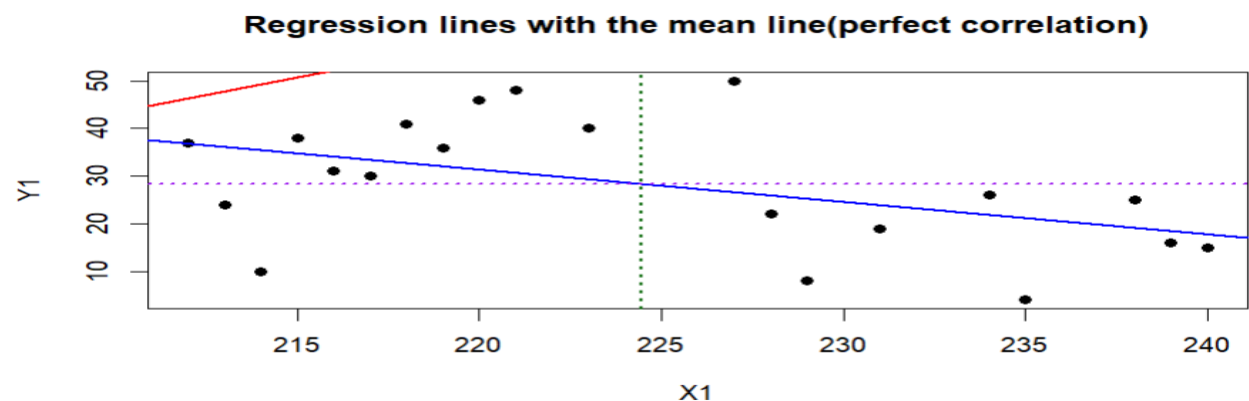
```
abline(a5, b5, col = "blue", lwd = 2)
```

```
abline(intercept_rotated, slope_rotated, col = "red", lwd = 2)
```

```
abline(v = mean(x3), col = "darkgreen", lwd = 2, lty = 3)
```

```
abline(h = mean(y3), col = "purple", lwd = 2, lty = 3)
```

Output - 11 :





PRACTICAL LIST – 5

5.1 Visualize multivariate data using plots and graphs in R.

5.2 Practical exercises to simple linear regression applying the matrix approach.

5.3 To test of significance of parameters fitted simple linear regression model.

5.4 Practical applications of the matrix approach in multiple regression.

5.5 Analysing and presenting results from real-world datasets in R.

5.1 Visualize multivariate data using plots and graphs in R

```
# Load dataset  
data(mtcars)
```

```
# View first few rows  
head(mtcars)
```

```
      mpg cyl  disp  hp drat   wt  qsec vs am gear carb  
Mazda RX4      21.0   6  160 110 3.90 2.620 16.46 0 1   4   4  
Mazda RX4 Wag  21.0   6  160 110 3.90 2.875 17.02 0 1   4   4  
Datsun 710      22.8   4  108  93 3.85 2.320 18.61 1 1   4   1  
Hornet 4 Drive  21.4   6  258 110 3.08 3.215 19.44 1 0   3   1  
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02 0 0   3   2  
Valiant         18.1   6  225 105 2.76 3.460 20.22 1 0   3   1
```

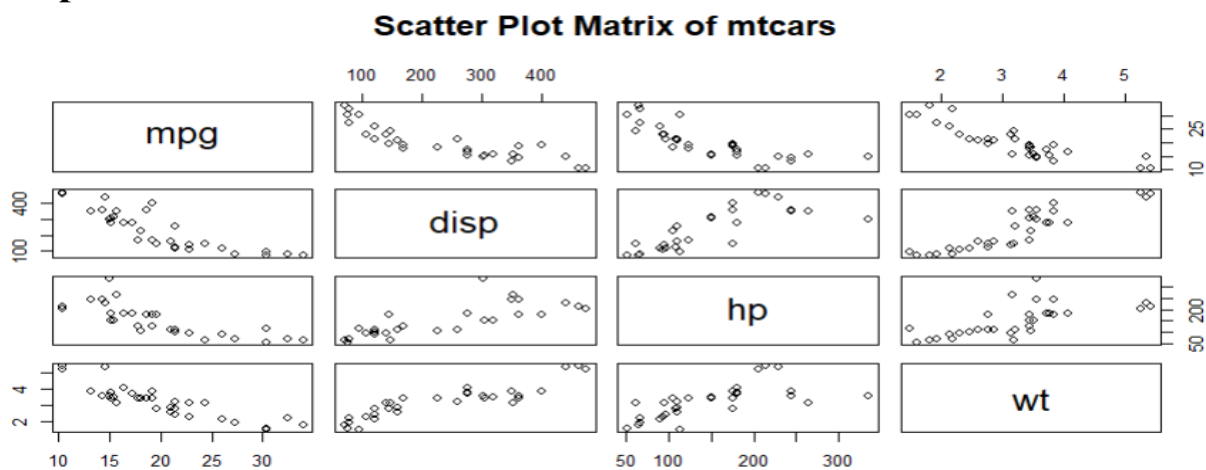
```
# Summary of dataset  
summary(mtcars)
```

```
      mpg      cyl      disp      hp      drat  
Min. :10.40 Min. :4.000 Min. : 71.1 Min. : 52.0 Min. :2.760  
1st Qu.:15.43 1st Qu.:4.000 1st Qu.:120.8 1st Qu.: 96.5 1st Qu.:3.080  
Median :19.20 Median :6.000 Median :196.3 Median :123.0 Median :3.695  
Mean :20.09 Mean :6.188 Mean :230.7 Mean :146.7 Mean :3.597  
3rd Qu.:22.80 3rd Qu.:8.000 3rd Qu.:326.0 3rd Qu.:180.0 3rd Qu.:3.920  
Max. :33.90 Max. :8.000 Max. :472.0 Max. :335.0 Max. :4.930  
      wt      qsec      vs      am      gear  
Min. :1.513 Min. :14.50 Min. :0.0000 Min. :0.0000 Min. :3.000  
1st Qu.:2.581 1st Qu.:16.89 1st Qu.:0.0000 1st Qu.:0.0000 1st Qu.:3.000  
Median :3.325 Median :17.71 Median :0.0000 Median :0.0000 Median :4.000  
Mean :3.217 Mean :17.85 Mean :0.4375 Mean :0.4062 Mean :3.688  
3rd Qu.:3.610 3rd Qu.:18.90 3rd Qu.:1.0000 3rd Qu.:1.0000 3rd Qu.:4.000  
Max. :5.424 Max. :22.90 Max. :1.0000 Max. :1.0000 Max. :5.000
```

```
carb
Min. :1.000
1st Qu.:2.000
Median :2.000
Mean :2.812
3rd Qu.:4.000
Max. :8.000
```

```
# Scatter plot matrix for multiple variables
pairs(mtcars[, c("mpg", "disp", "hp", "wt")], main = "Scatter Plot Matrix of mtcars")
```

Output - 1 :

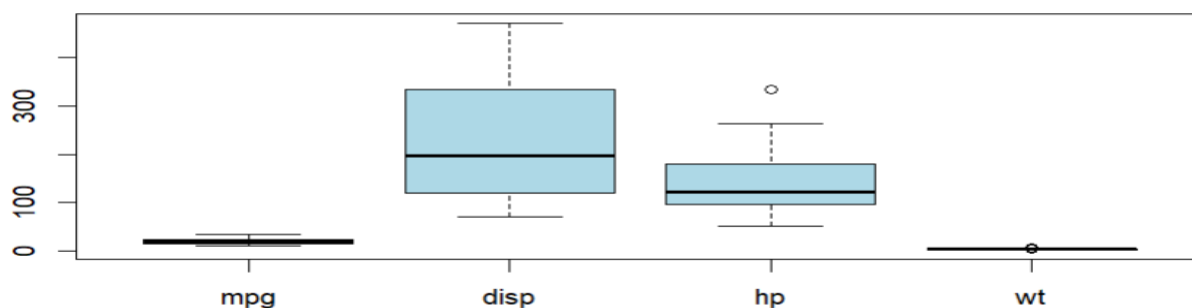


```
# Correlation matrix
cor(mtcars[, c("mpg", "disp", "hp", "wt")])
      mpg      disp      hp      wt
mpg  1.0000000 -0.8475514 -0.7761684 -0.8676594
disp -0.8475514  1.0000000  0.7909486  0.8879799
hp   -0.7761684  0.7909486  1.0000000  0.6587479
wt   -0.8676594  0.8879799  0.6587479  1.0000000
```

```
# Bo-xplot for selected variables
boxplot(mtcars[, c("mpg", "disp", "hp", "wt")], main = "Boxplot of Selected Variables", col = "lightblue")
```

Output - 2 :

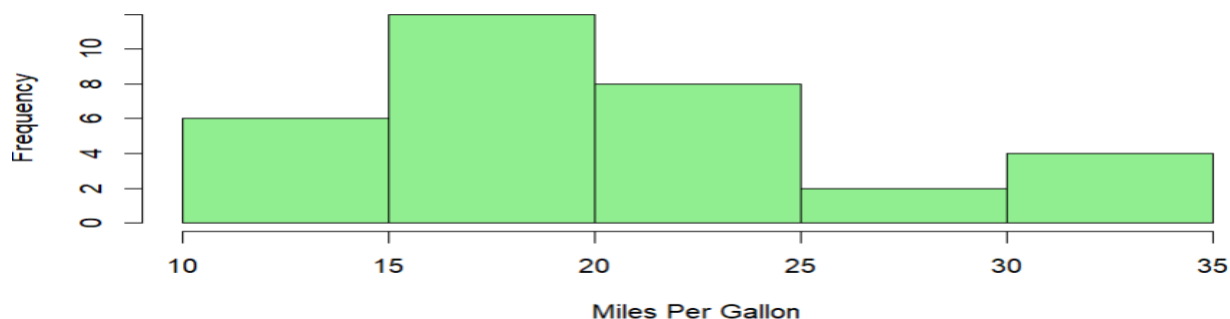
Boxplot of Selected Variables



```
# Histogram of mpg
hist(mtcars$mpg,main = "Histogram of MPG",xlab = "Miles Per Gallon",col = "lightgreen",
border = "black")
```

Output - 3 :

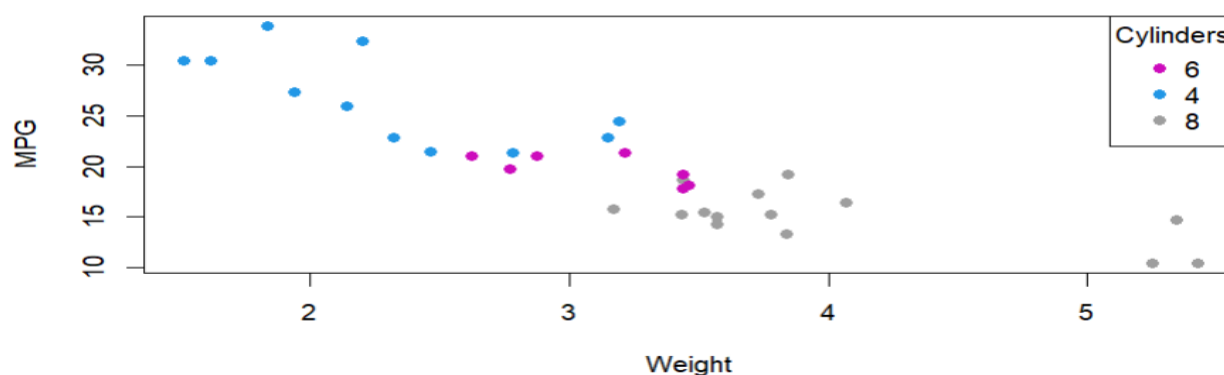
Histogram of MPG



```
# Scatter plot with color based on number of cylinders
plot(mtcars$wt, mtcars$mpg,col = mtcars$cyl,pch = 19,xlab = "Weight",ylab = "MPG",main =
"MPG vs Weight (colored by cylinders)")
```

Output - 4 :

MPG vs Weight (colored by cylinders)





```
legend("topright",legend = unique(mtcars$cyl),col = unique(mtcars$cyl),pch = 19,title =  
"Cylinders")
```

5.2 Simple Linear Regression using Matrix Approach

```
# Load dataset  
data(mtcars)
```

```
# Response variable  
Y = as.matrix(mtcars$mpg)
```

```
# Predictor variable with intercept  
X = as.matrix(cbind(1, mtcars$wt))
```

```
# Estimate coefficients using matrix formula  
beta_hat = solve(t(X) %*% X) %*% t(X) %*% Y
```

```
cat("Estimated coefficients using matrix approach:\n")
```

```
    Estimated coefficients using matrix approach:
```

```
print(beta_hat)
```

```
    [,1]  
[1,] 37.285126  
[2,] -5.344472
```

```
# Predicted values  
Y_hat = X %*% beta_hat
```

```
# Residuals  
residuals = Y - Y_hat
```

```
cat("First 6 predicted values:\n")
```

```
    First 6 predicted values:
```

```
print(head(Y_hat))
```

```
    [,1]  
[1,] 23.28261  
[2,] 21.91977  
[3,] 24.88595  
[4,] 20.10265  
[5,] 18.90014  
[6,] 18.79325
```

```
cat("First 6 residuals:\n")
```

```
    First 6 residuals:
```

```
print(head(residuals))
```

```
    [,1]
```



```
[1,] -2.2826106  
[2,] -0.9197704  
[3,] -2.0859521  
[4,]  1.2973499  
[5,] -0.2001440  
[6,] -0.6932545
```

```
# Compare with lm() function  
model = lm(mpg ~ wt, data = mtcars)
```

```
cat("Coefficients using lm():\n")  
      Coefficients using lm():  
print(coef(model))  
(Intercept)      wt  
  37.285126  -5.344472
```

```
# 5.3 Test significance of parameters in simple linear regression
```

```
# Load dataset  
data(mtcars)
```

```
# Fit model using lm()  
model = lm(mpg ~ wt, data = mtcars)
```

```
# Model summary  
summary(model)
```

Call:

```
lm(formula = mpg ~ wt, data = mtcars)
```

Residuals:

```
      Min       1Q   Median       3Q      Max  
-4.5432 -2.3647 -0.1252  1.4096  6.8727
```

Coefficients:

```
      Estimate Std. Error t value Pr(>|t|)  
(Intercept)  37.2851    1.8776  19.858 < 2e-16 ***  
wt           -5.3445    0.5591  -9.559 1.29e-10 ***  
---  
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 3.046 on 30 degrees of freedom  
Multiple R-squared:  0.7528, Adjusted R-squared:  0.7446  
F-statistic: 91.38 on 1 and 30 DF, p-value: 1.294e-10
```



5.4 Multiple Regression using Matrix Approach

Load dataset

```
data(mtcars)
```

Response variable

```
Y = as.matrix(mtcars$mpg)
```

Predictor matrix with intercept

```
X = as.matrix(cbind(1, mtcars$wt, mtcars$hp, mtcars$disp))
```

Estimate coefficients

```
beta_hat = solve(t(X) %*% X) %*% t(X) %*% Y
```

```
cat("Estimated coefficients using matrix approach:\n")
```

Estimated coefficients using matrix approach:

```
print(beta_hat)
```

[,1]

[1,] 37.1055052690

[2,] -3.8008905826

[3,] -0.0311565508

[4,] -0.0009370091

Predicted values

```
Y_hat = X %*% beta_hat
```

Residuals

```
e = Y - Y_hat
```

```
cat("First 6 predicted values:\n")
```

First 6 predicted values:

```
print(head(Y_hat))
```

[,1]

[1,] 23.57003

[2,] 22.60080

[3,] 25.28868

[4,] 21.21667

[5,] 18.24072

[6,] 20.47216

```
cat("First 6 residuals:\n")
```

First 6 residuals:

```
print(head(e))
```

[,1]

[1,] -2.5700299

[2,] -1.6008028



```
[3,] -2.4886829  
[4,] 0.1833269  
[5,] 0.4592780  
[6,] -2.3721590
```

```
# Compare with lm()  
model_multi = lm(mpg ~ wt + hp + disp, data = mtcars)  
  
cat("Coefficients using lm():\n")  
Coefficients using lm():  
print(coef(model_multi))  
(Intercept)      wt      hp      disp  
37.1055052690 -3.8008905826 -0.0311565508 -0.0009370091
```

```
# Full summary  
summary(model_multi)  
Call:  
lm(formula = mpg ~ wt + hp + disp, data = mtcars)
```

```
Residuals:  
    Min     1Q  Median     3Q     Max  
-3.891 -1.640 -0.172  1.061  5.861
```

```
Coefficients:  
            Estimate Std. Error t value Pr(>|t|)  
(Intercept) 37.105505  2.110815  17.579 < 2e-16 ***  
wt          -3.800891  1.066191  -3.565 0.00133 **  
hp          -0.031157  0.011436  -2.724 0.01097 *  
disp        -0.000937  0.010350  -0.091 0.92851
```

```
---  
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 2.639 on 28 degrees of freedom  
Multiple R-squared:  0.8268, Adjusted R-squared:  0.8083  
F-statistic: 44.57 on 3 and 28 DF, p-value: 8.65e-11
```

5.5 Analysing and presenting results from real-world datasets in R

```
# Load dataset  
data(mtcars)
```

```
# Display structure and summary  
str(mtcars)  
'data.frame':  32 obs. of  11 variables:  
 $ mpg : num  21 21 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 ...  
 $ cyl : num  6 6 4 6 8 6 8 4 4 6 ...
```



```
$ disp: num 160 160 108 258 360 ...
$ hp : num 110 110 93 110 175 105 245 62 95 123 ...
$ drat: num 3.9 3.9 3.85 3.08 3.15 2.76 3.21 3.69 3.92 3.92 ...
$ wt : num 2.62 2.88 2.32 3.21 3.44 ...
$ qsec: num 16.5 17 18.6 19.4 17 ...
$ vs : num 0 0 1 1 0 1 0 1 1 1 ...
$ am : num 1 1 1 0 0 0 0 0 0 0 ...
$ gear: num 4 4 4 3 3 3 3 4 4 4 ...
$ carb: num 4 4 1 1 2 1 4 2 2 4 ...
```

```
summary(mtcars)
```

```
      mpg      cyl      disp      hp      drat
Min. :10.40 Min. :4.000 Min. :71.1 Min. :52.0 Min. :2.760
1st Qu.:15.43 1st Qu.:4.000 1st Qu.:120.8 1st Qu.:96.5 1st Qu.:3.080
Median :19.20 Median :6.000 Median :196.3 Median :123.0 Median :3.695
Mean :20.09 Mean :6.188 Mean :230.7 Mean :146.7 Mean :3.597
3rd Qu.:22.80 3rd Qu.:8.000 3rd Qu.:326.0 3rd Qu.:180.0 3rd Qu.:3.920
Max. :33.90 Max. :8.000 Max. :472.0 Max. :335.0 Max. :4.930

      wt      qsec      vs      am      gear
Min. :1.513 Min. :14.50 Min. :0.0000 Min. :0.0000 Min. :3.000
1st Qu.:2.581 1st Qu.:16.89 1st Qu.:0.0000 1st Qu.:0.0000 1st Qu.:3.000
Median :3.325 Median :17.71 Median :0.0000 Median :0.0000 Median :4.000
Mean :3.217 Mean :17.85 Mean :0.4375 Mean :0.4062 Mean :3.688
3rd Qu.:3.610 3rd Qu.:18.90 3rd Qu.:1.0000 3rd Qu.:1.0000 3rd Qu.:4.000
Max. :5.424 Max. :22.90 Max. :1.0000 Max. :1.0000 Max. :5.000

      carb
Min. :1.000
1st Qu.:2.000
Median :2.000
Mean :2.812
3rd Qu.:4.000
Max. :8.000
```

```
# Fit multiple regression model
model = lm(mpg ~ wt + hp + disp, data = mtcars)
```

```
# Show regression summary
summary(model)
```

Call:

```
lm(formula = mpg ~ wt + hp + disp, data = mtcars)
```

Residuals:

```
      Min      1Q  Median      3Q      Max
-3.891 -1.640 -0.172  1.061  5.861
```

Coefficients:



```
      Estimate Std. Error t value Pr(>|t|)
(Intercept) 37.105505  2.110815 17.579 < 2e-16 ***
wt          -3.800891  1.066191 -3.565 0.00133 **
hp          -0.031157  0.011436 -2.724 0.01097 *
disp        -0.000937  0.010350 -0.091 0.92851
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 2.639 on 28 degrees of freedom
Multiple R-squared: 0.8268, Adjusted R-squared: 0.8083
F-statistic: 44.57 on 3 and 28 DF, p-value: 8.65e-11

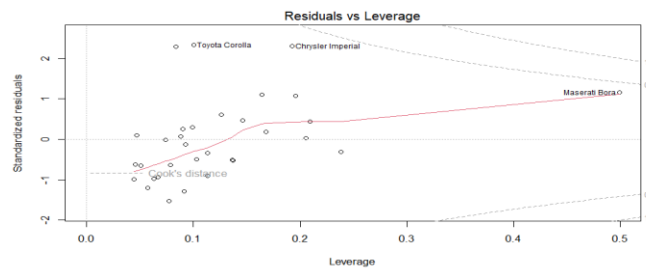
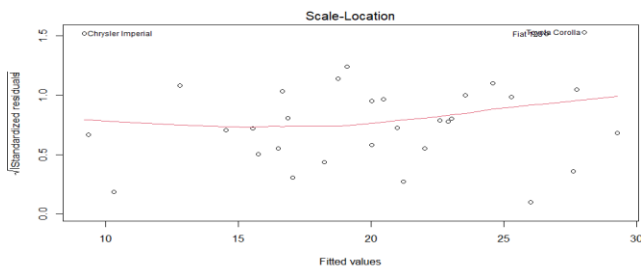
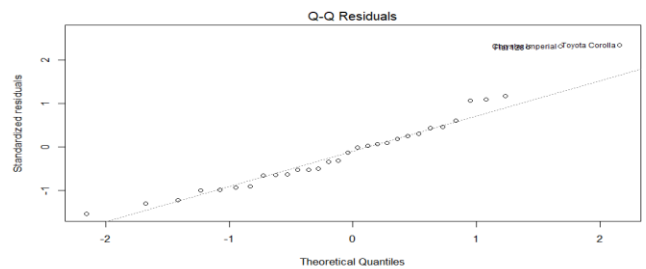
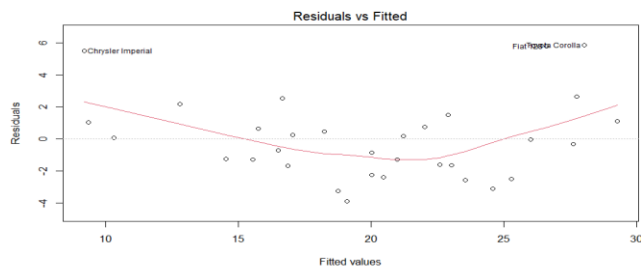
```
# Confidence intervals
confint(model)
      2.5 %    97.5 %
(Intercept) 32.78169625 41.429314293
wt          -5.98488310 -1.616898063
hp          -0.05458171 -0.007731388
disp        -0.02213750 0.020263482
```

```
# Diagnostic plots
par(mfrow = c(2, 2))
plot(model)
```

```
# Actual vs Predicted plot
predicted <- predict(model)
```

```
plot(mtcars$mpg, predicted, xlab = "Actual MPG", ylab = "Predicted MPG", main = "Actual vs
Predicted MPG", pch = 19, col = "blue")
abline(0, 1, lwd = 2, col = "red")
```

Output - 5 :



Residual plot

```
plot(predicted, resid(model), xlab = "Predicted Values", ylab = "Residuals", main = "Residual Plot", pch = 19, col = "darkgreen")
abline(h = 0, lwd = 2, col = "red")
```

Output - 6 :

